

# Unsupervised Learning: Clustering Algorithms

Panthadeep Bhattacharjee  
Dept. of CSE, NIT Rourkela

*panthadeep.edu@gmail.com*



# Clustering

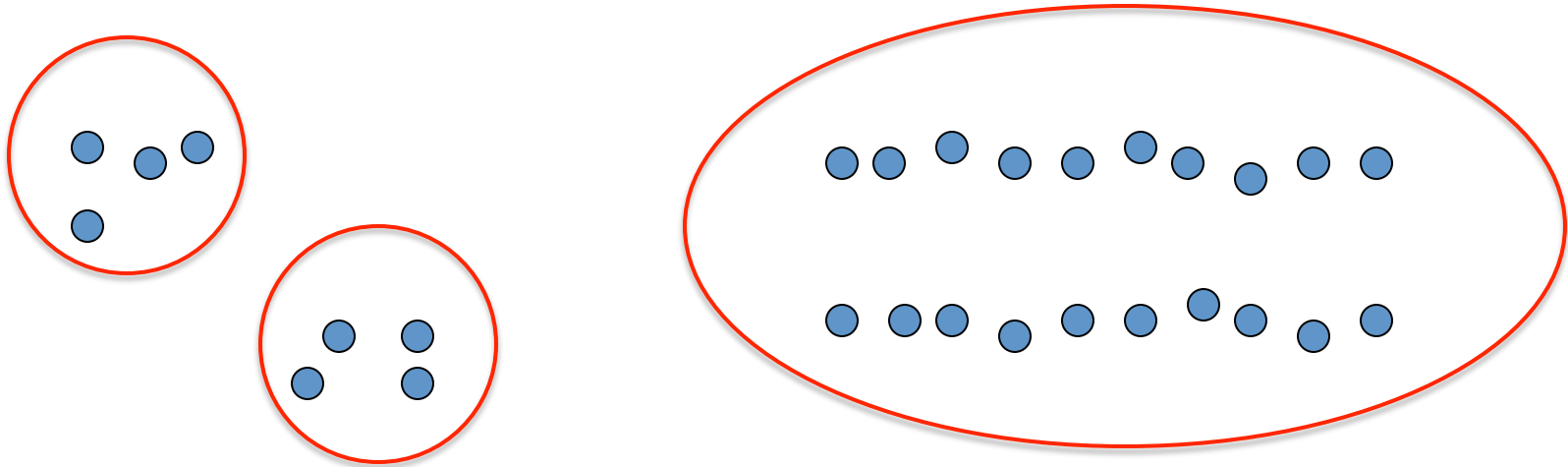
## Clustering:

- **Unsupervised learning**
- Requires data, but no labels
- **Detect patterns** e.g. in
  - Group emails or search results
  - Customer shopping patterns
  - Regions of images
- Useful when don't know what you're looking for
- But: can get gibberish



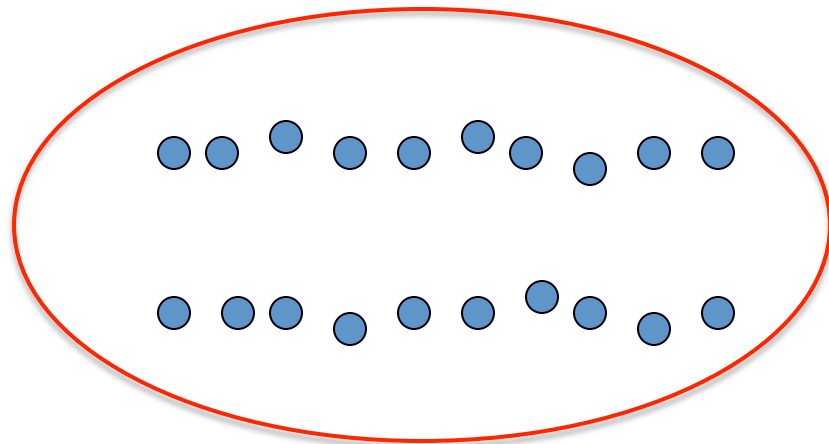
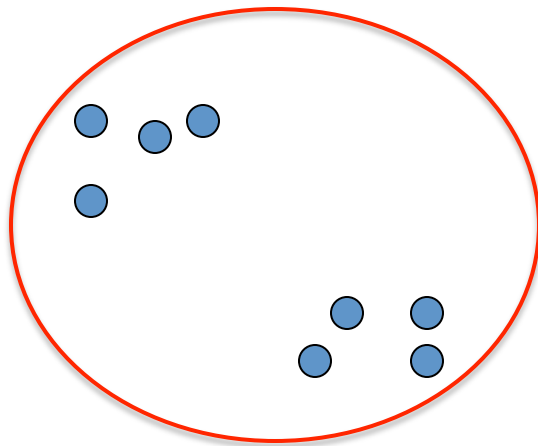
# Clustering

- **Basic idea:** group together similar instances
- **Example:** 2D point patterns



# Clustering

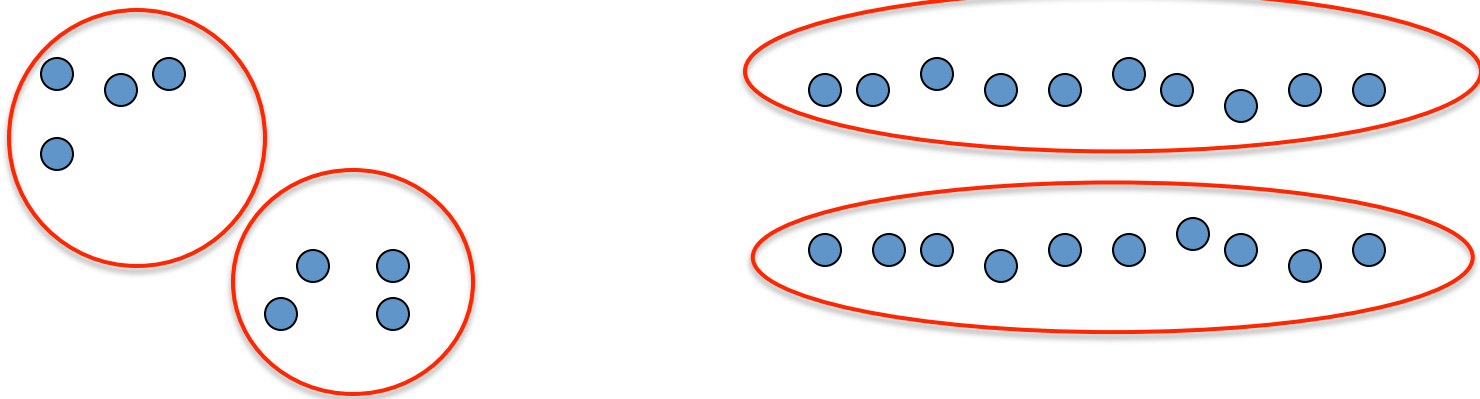
- **Basic idea:** group together similar instances
- **Example:** 2D point patterns





# Clustering

- **Basic idea:** group together similar instances
- **Example:** 2D point patterns

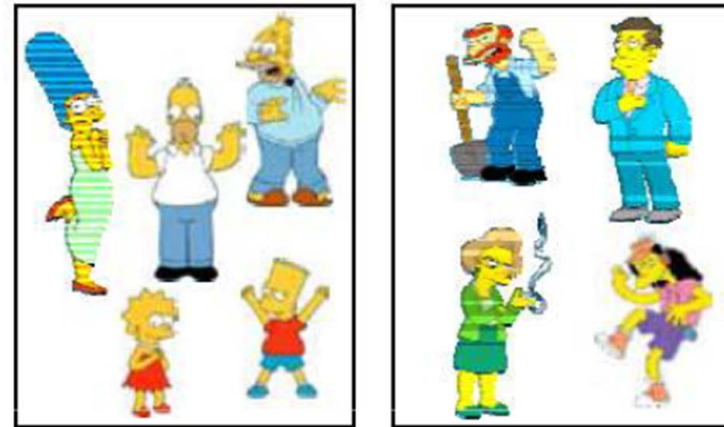


- **What could “similar” mean?**
  - One option: small Euclidean distance (squared)
$$\text{dist}(\vec{x}, \vec{y}) = ||\vec{x} - \vec{y}||_2^2$$
  - Clustering results are crucially dependent on the measure of similarity (or distance) between “points” to be clustered

# Clustering algorithms

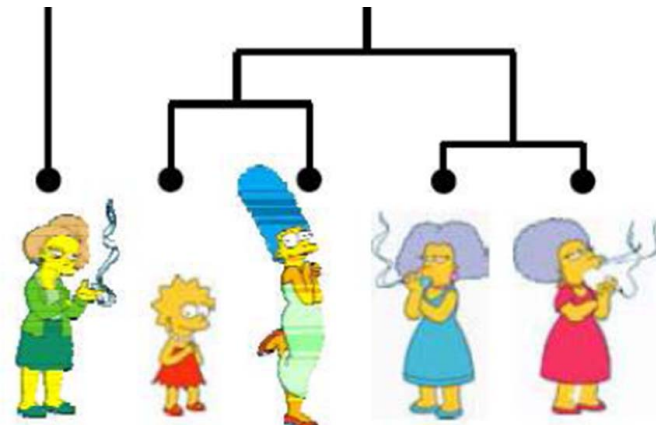
- Partition algorithms (Flat)

- K-means
- Mixture of Gaussian
- Spectral Clustering



- Hierarchical algorithms

- Bottom up – agglomerative
- Top down – divisive



# Clustering examples

## Image segmentation

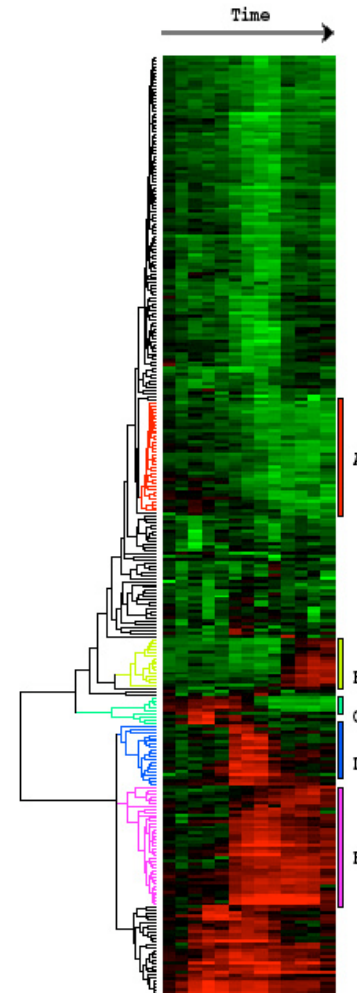
Goal: Break up the image into meaningful or perceptually similar regions



[Slide from James Hayes]

# Clustering examples

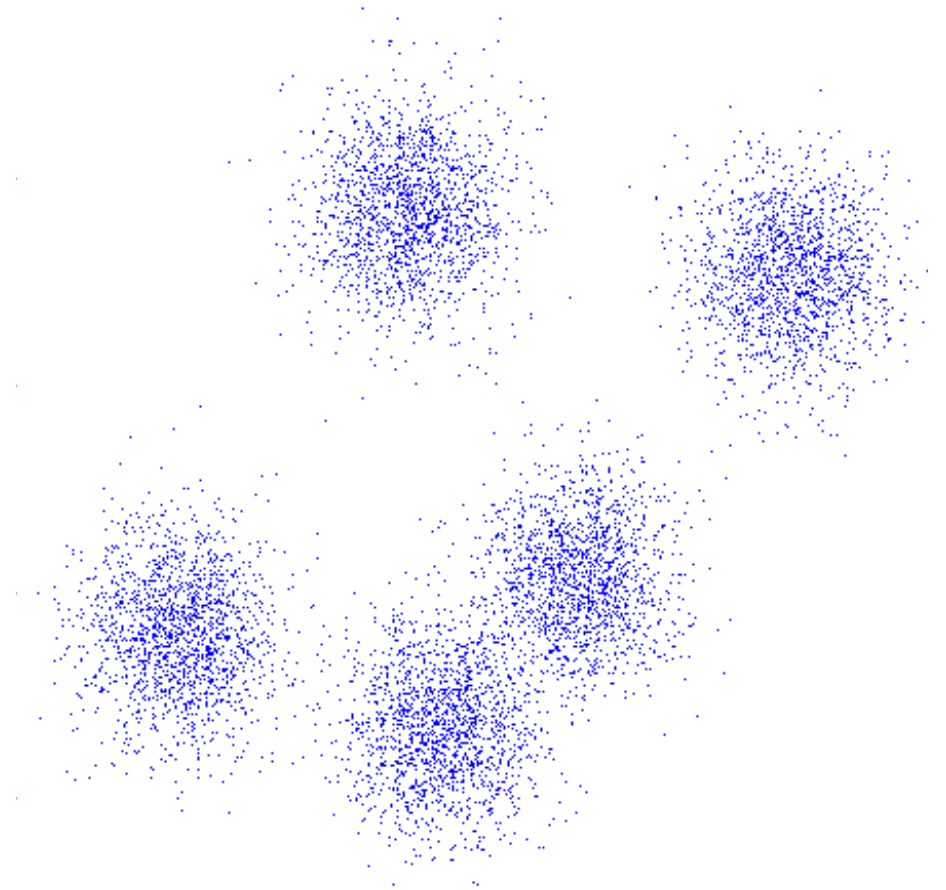
## Clustering gene expression data



Eisen et al, PNAS 1998

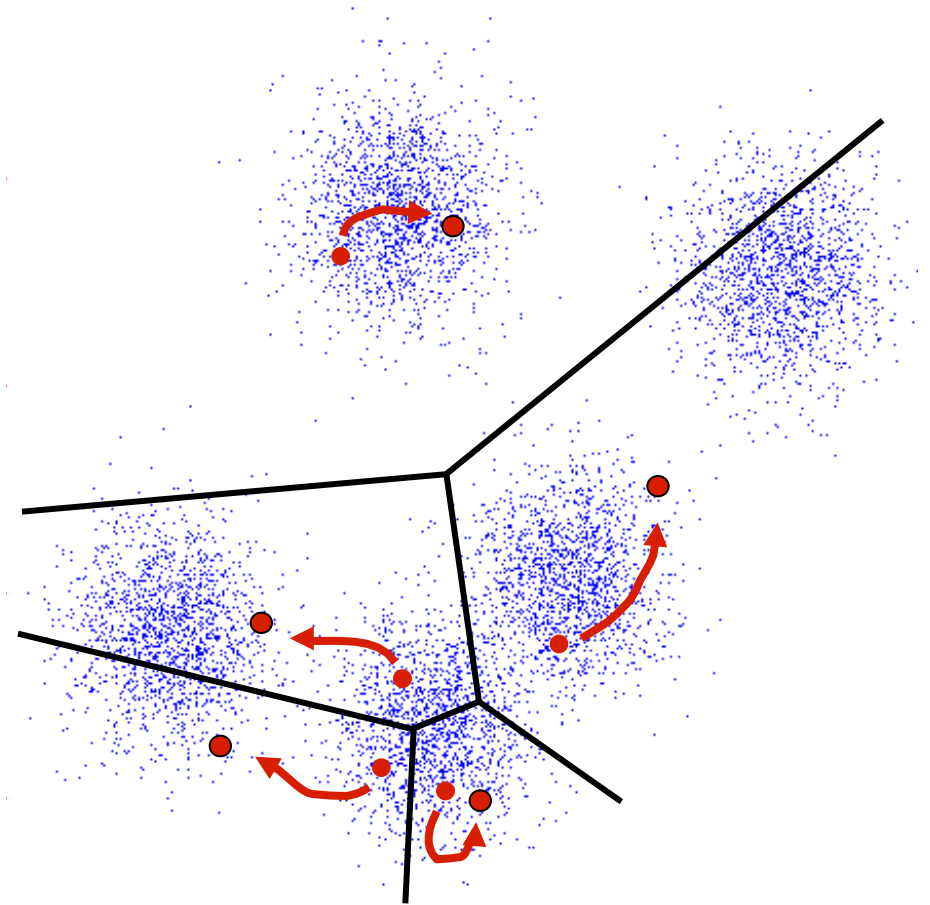
# K-Means

- An iterative clustering algorithm
  - **Initialize:** Pick  $K$  random points as cluster centers
  - **Alternate:**
    1. Assign data points to closest cluster center
    2. Change the cluster center to the average of its assigned points
  - **Stop** when no points' assignments change

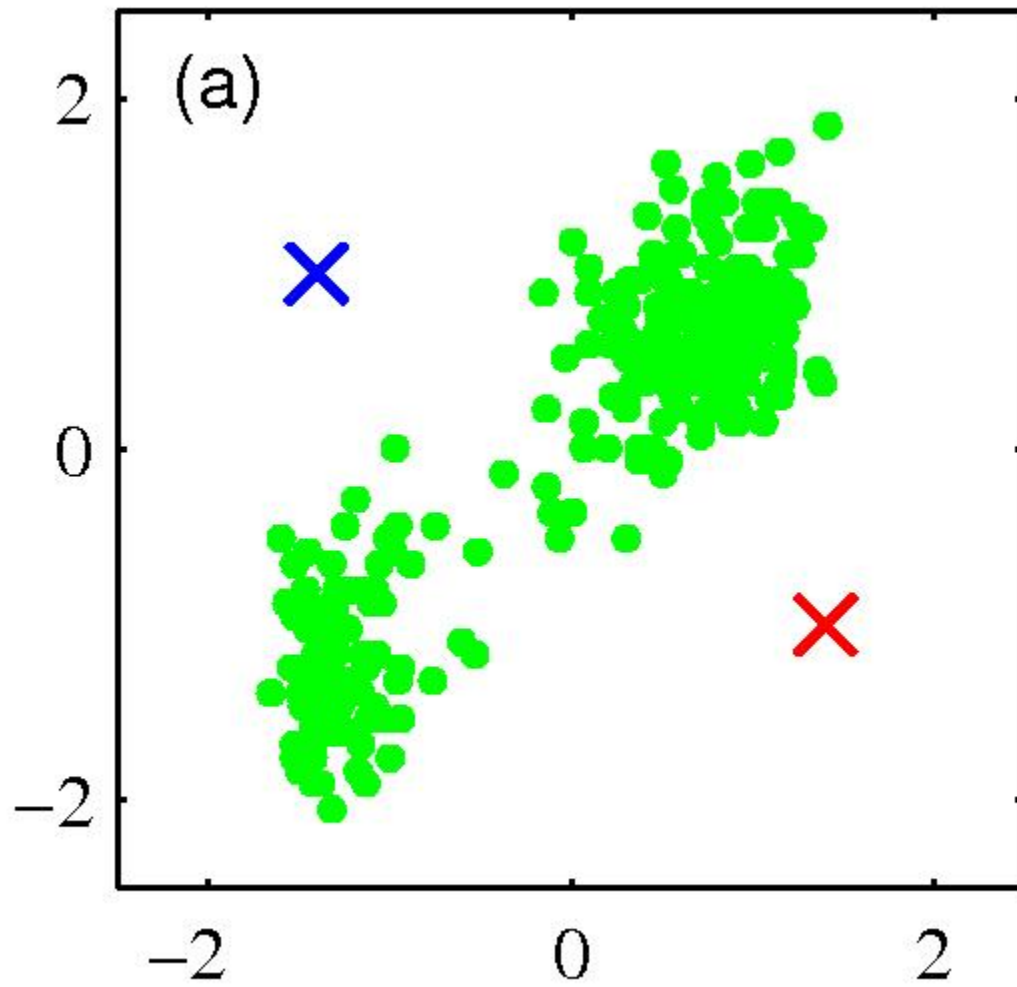


# K-Means

- An iterative clustering algorithm
  - **Initialize:** Pick  $K$  random points as cluster centers
  - **Alternate:**
    1. Assign data points to closest cluster center
    2. Change the cluster center to the average of its assigned points
  - **Stop** when no points' assignments change



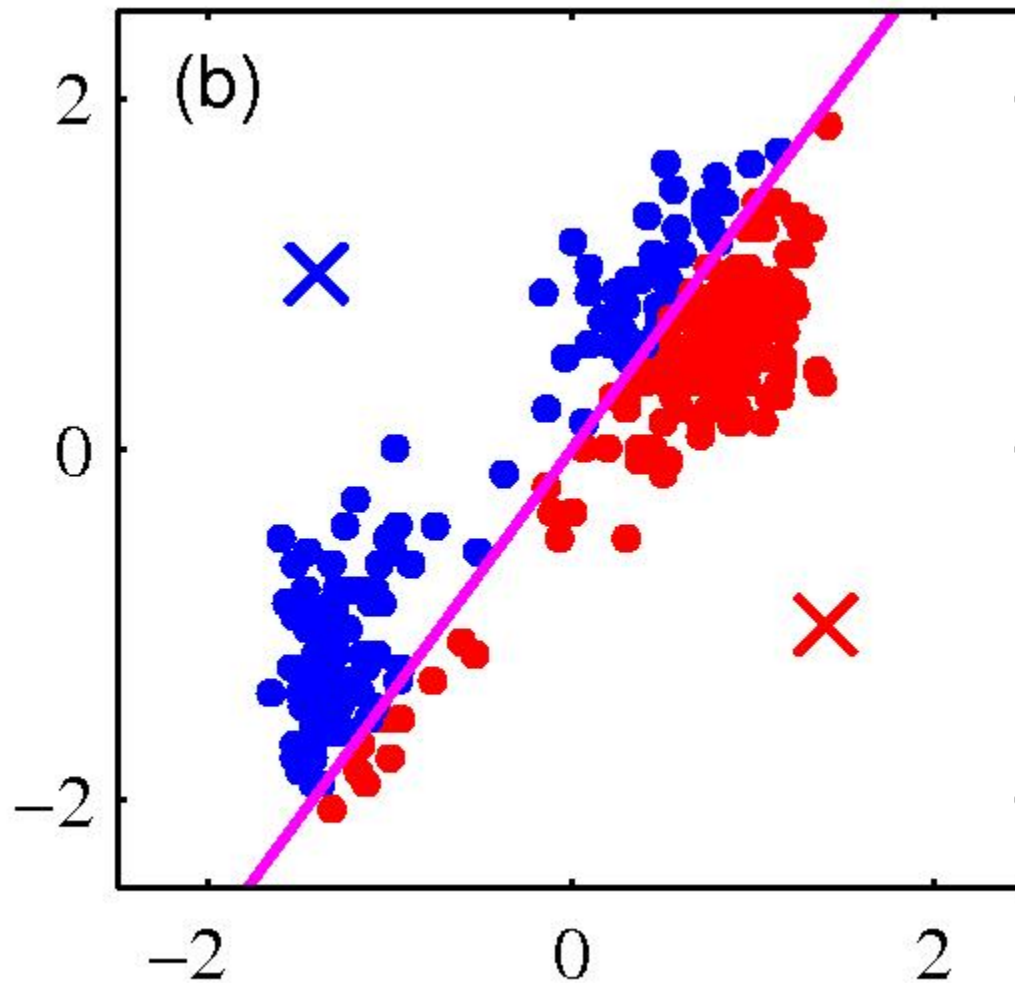
# K-means clustering: Example



- Pick  $K$  random points as cluster centers (means)

Shown here for  $K=2$

# K-means clustering: Example

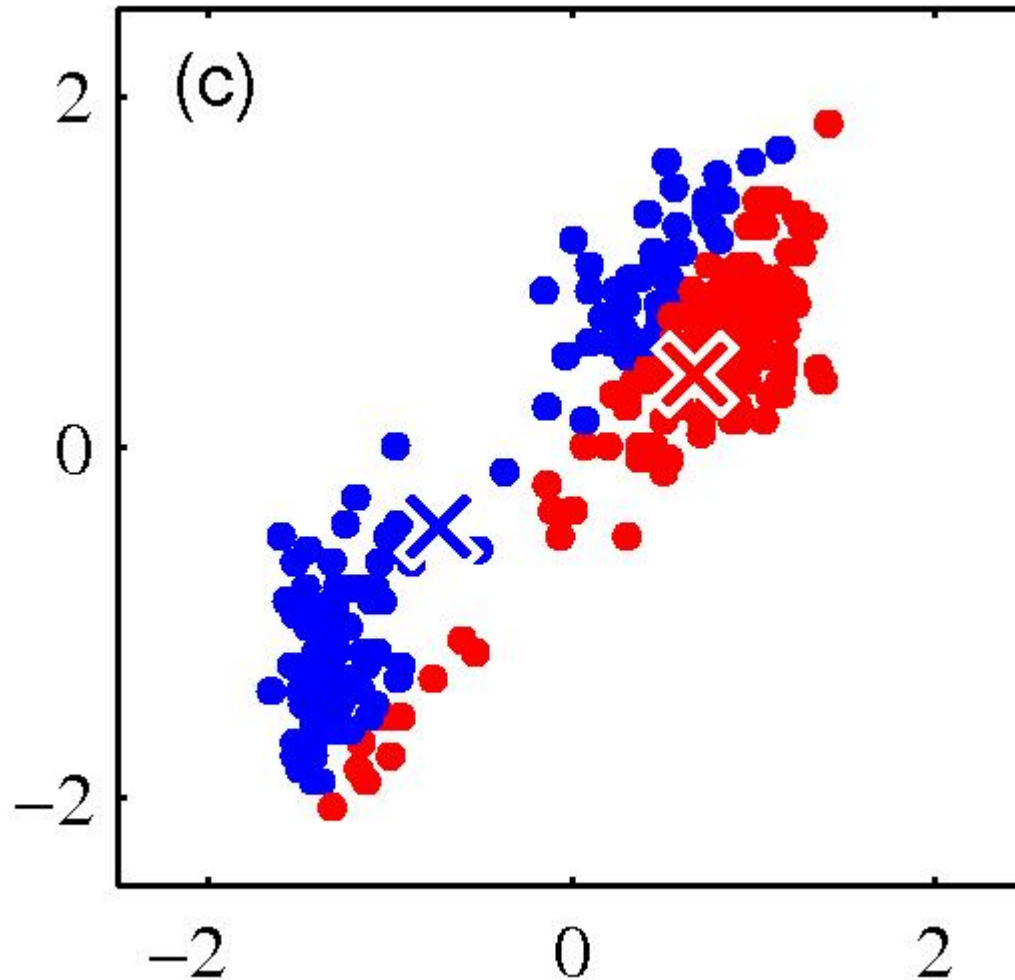


## Iterative Step 1

- Assign data points to closest cluster center



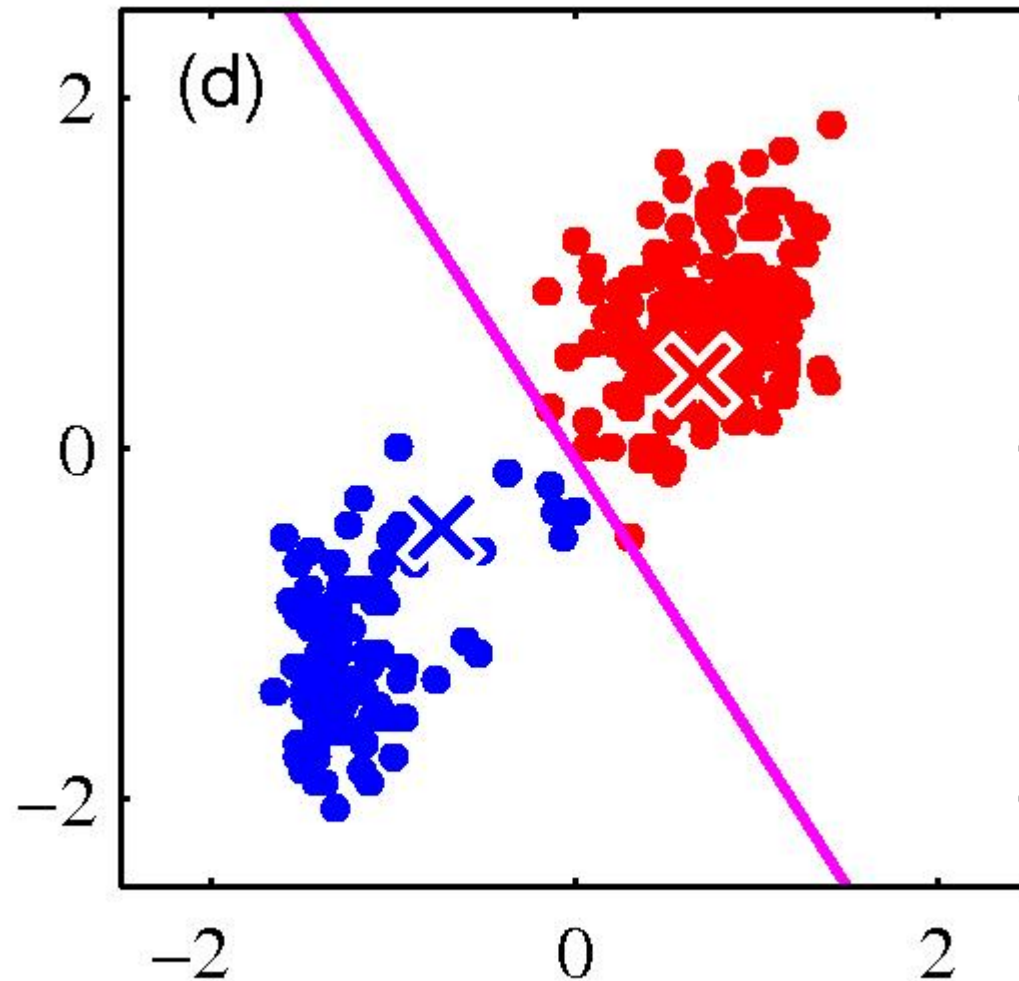
# K-means clustering: Example



## Iterative Step 2

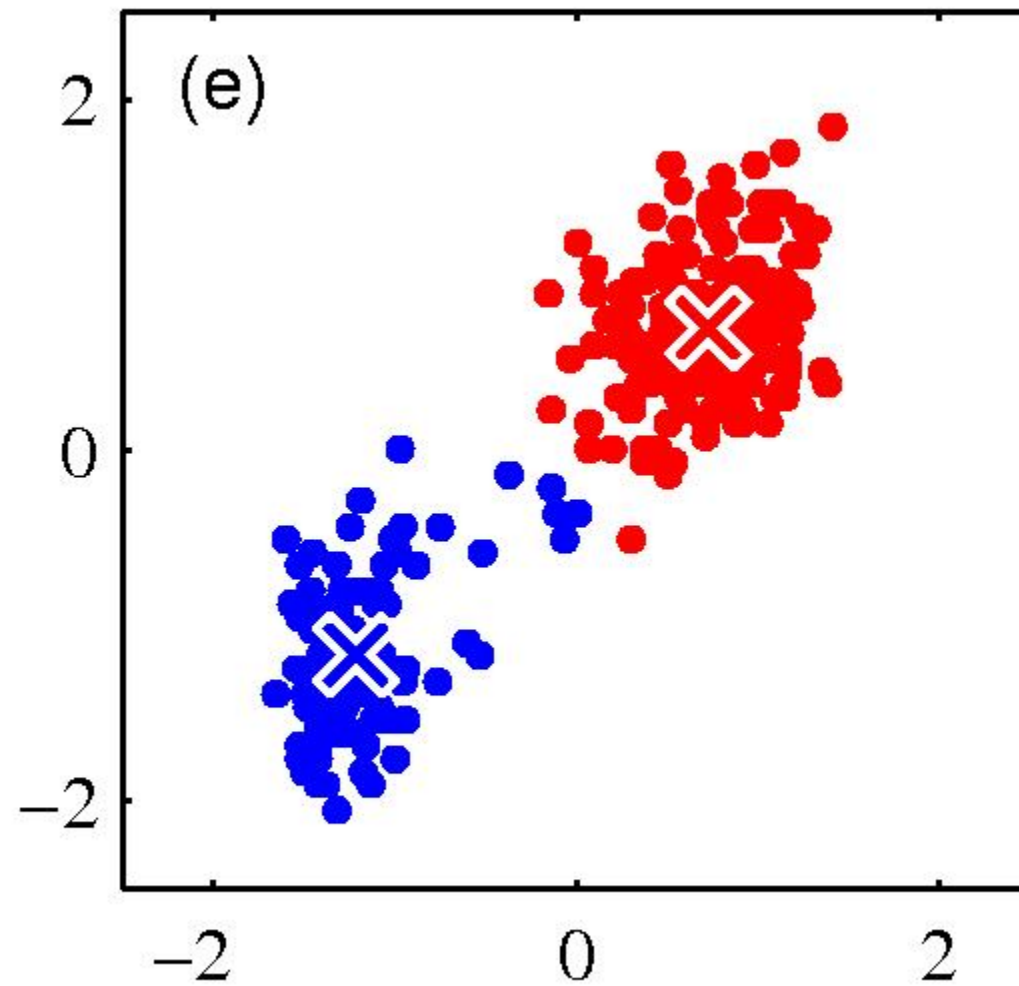
- Change the cluster center to the average of the assigned points

# K-means clustering: Example

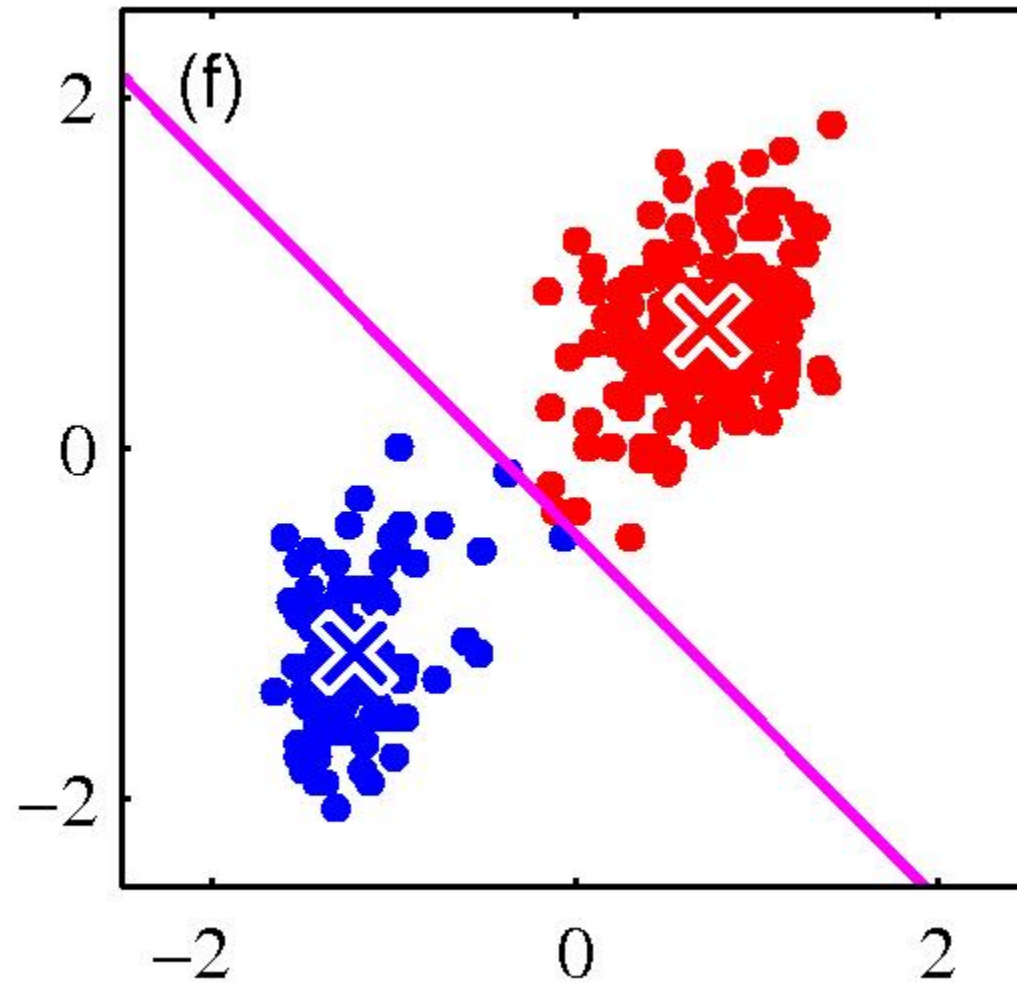


- Repeat until convergence

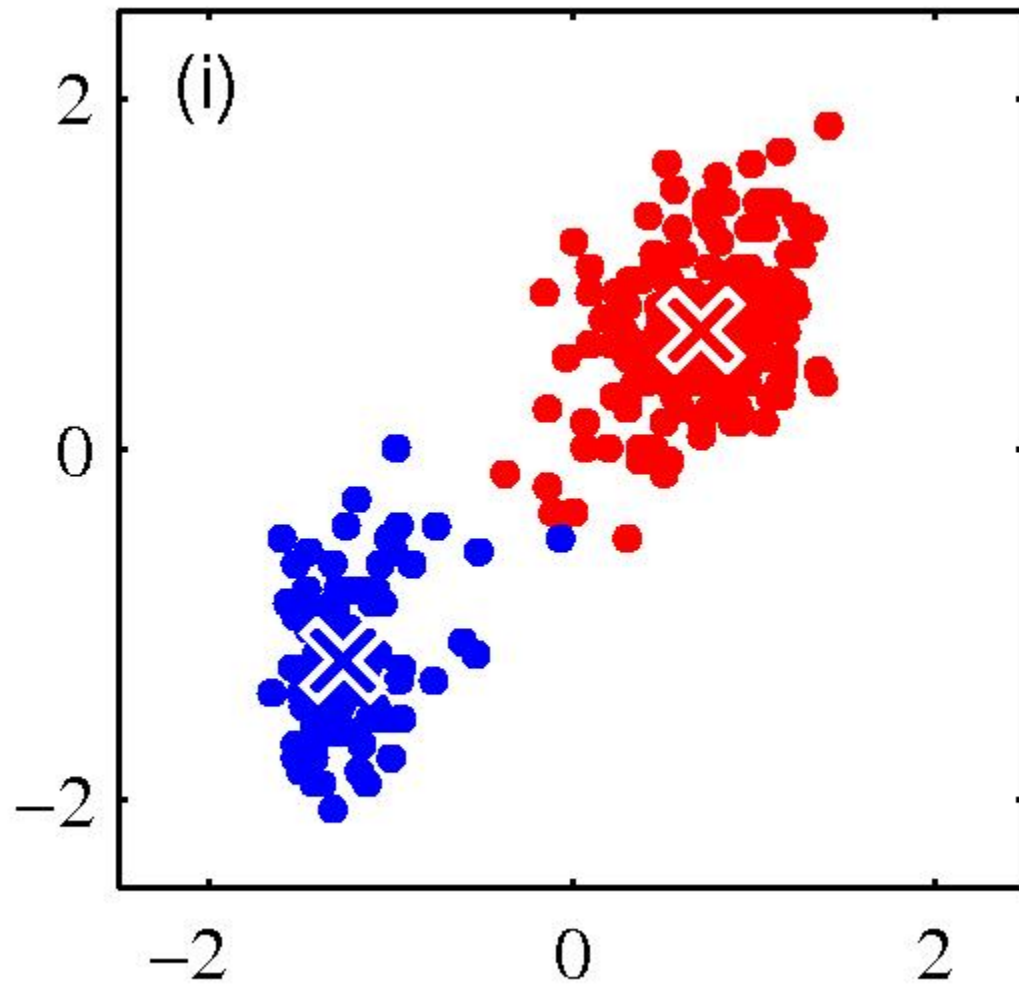
# K-means clustering: Example



# K-means clustering: Example



# K-means clustering: Example



# Properties of K-means **algorithm**

- Guaranteed to converge in a finite number of iterations
- Running time per iteration:
  1. Assign data points to closest cluster center  
 $O(KN)$  time
  2. Change the cluster center to the average of its assigned points  
 $O(N)$

# What properties should a distance measure have?

- Symmetric
  - $D(A,B)=D(B,A)$
  - Otherwise, we can say A looks like B but B does not look like A
- Positivity, and self-similarity
  - $D(A,B) \geq 0$ , and  $D(A,B)=0$  iff  $A=B$
  - Otherwise there will different objects that we cannot tell apart
- Triangle inequality
  - $D(A,B)+D(B,C) \geq D(A,C)$
  - Otherwise one can say “A is like B, B is like C, but A is not like C at all”

# Kmeans Convergence

## Objective

$$\min_{\mu} \min_C \sum_{i=1}^k \sum_{x \in C_i} |x - \mu_i|^2$$

1. Fix  $\mu$ , optimize  $C$ :

$$\min_C \sum_{i=1}^k \sum_{x \in C_i} |x - \mu_i|^2 = \min_c \sum_i^n |x_i - \mu_{x_i}|^2$$

***Step 1 of kmeans***

2. Fix  $C$ , optimize  $\mu$ :

$$\min_{\mu} \sum_{i=1}^k \sum_{x \in C_i} |x - \mu_i|^2$$

- Take partial derivative of  $\mu_i$  and set to zero, we have

$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

***Step 2 of kmeans***

Kmeans takes an alternating optimization approach, each step is guaranteed to decrease the objective – thus guaranteed to converge



# Example: K-Means for Segmentation

K=2



**Goal of Segmentation is to partition an image into regions each of which has reasonably homogenous visual appearance.**

Original



# Example: K-Means for Segmentation

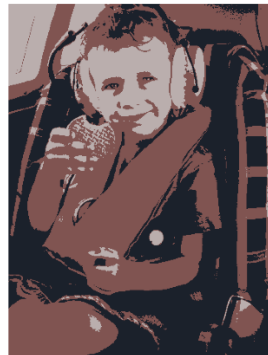
K=2



K=3



Original



# Example: K-Means for Segmentation

K=2



K=3



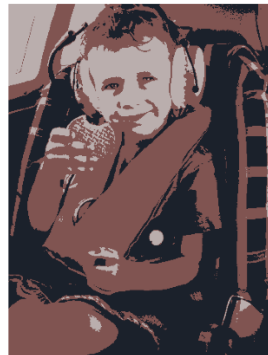
K=10



Original



4%



8%



17%



# Example: Vector quantization

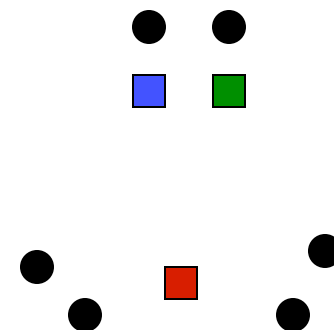
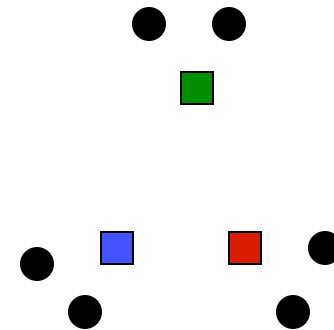


**FIGURE 14.9.** *Sir Ronald A. Fisher (1890 – 1962) was one of the founders of modern day statistics, to whom we owe maximum-likelihood, sufficiency, and many other fundamental concepts. The image on the left is a  $1024 \times 1024$  grayscale image at 8 bits per pixel. The center image is the result of  $2 \times 2$  block VQ, using 200 code vectors, with a compression rate of 1.9 bits/pixel. The right image uses only four code vectors, with a compression rate of 0.50 bits/pixel*

[Figure from Hastie *et al.* book]

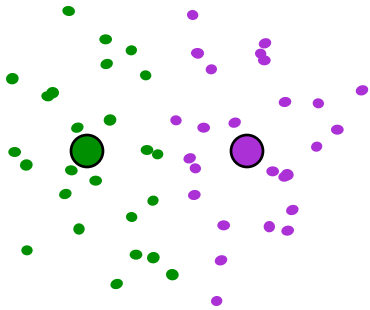
# Initialization

- K-means **algorithm** is a heuristic
  - Requires initial means
  - It does matter what you pick!
  - What can go wrong?
  - Various schemes for preventing this kind of thing: variance-based split / merge, initialization heuristics

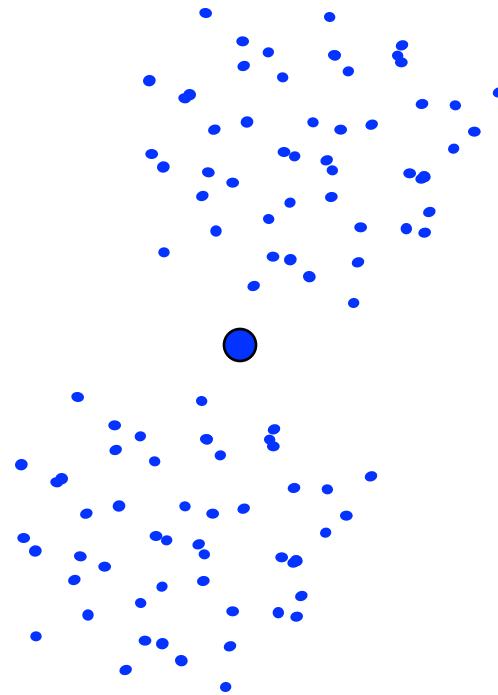


# K-Means Getting Stuck

A local optimum:

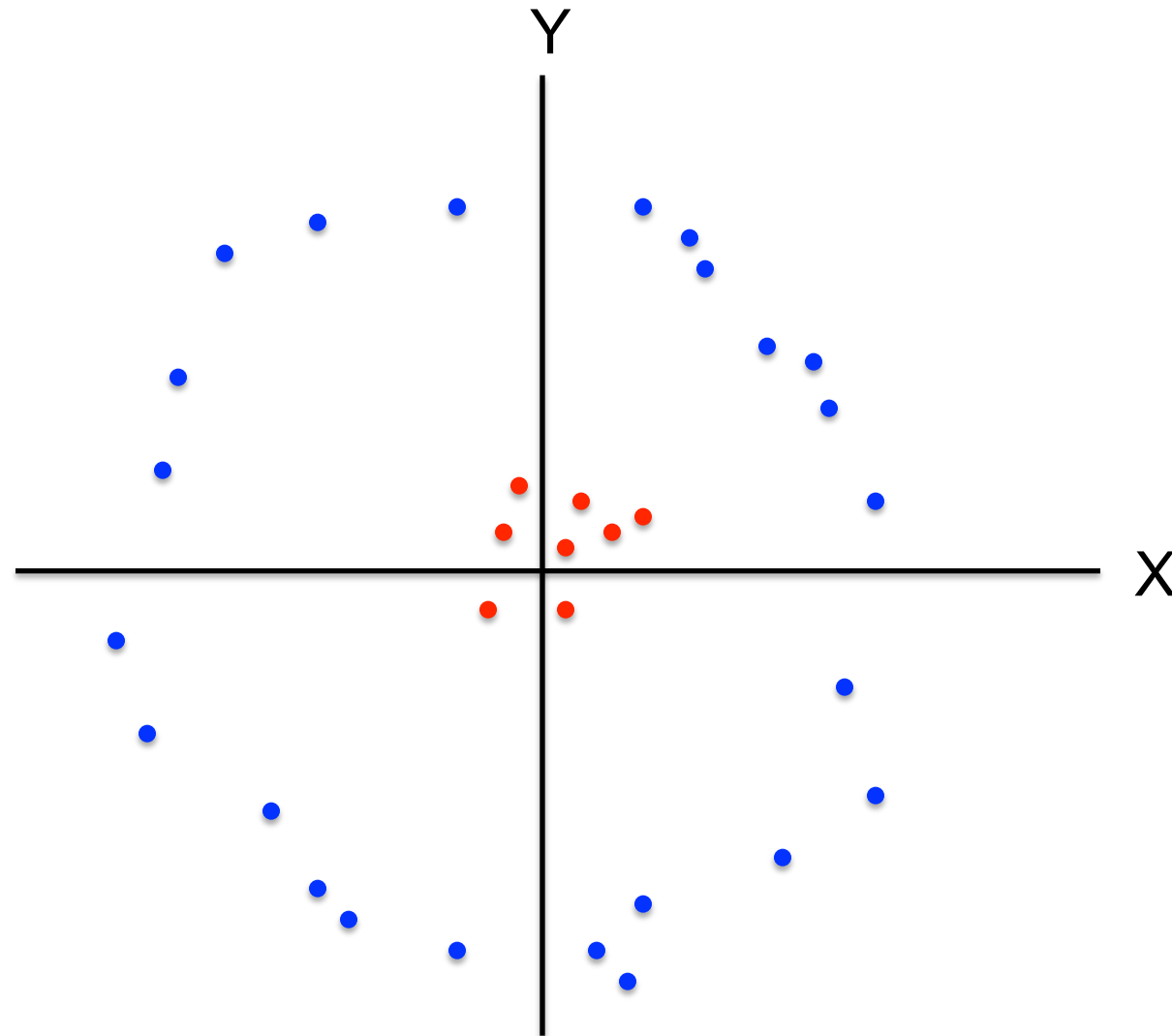


Would be better to have  
one cluster here

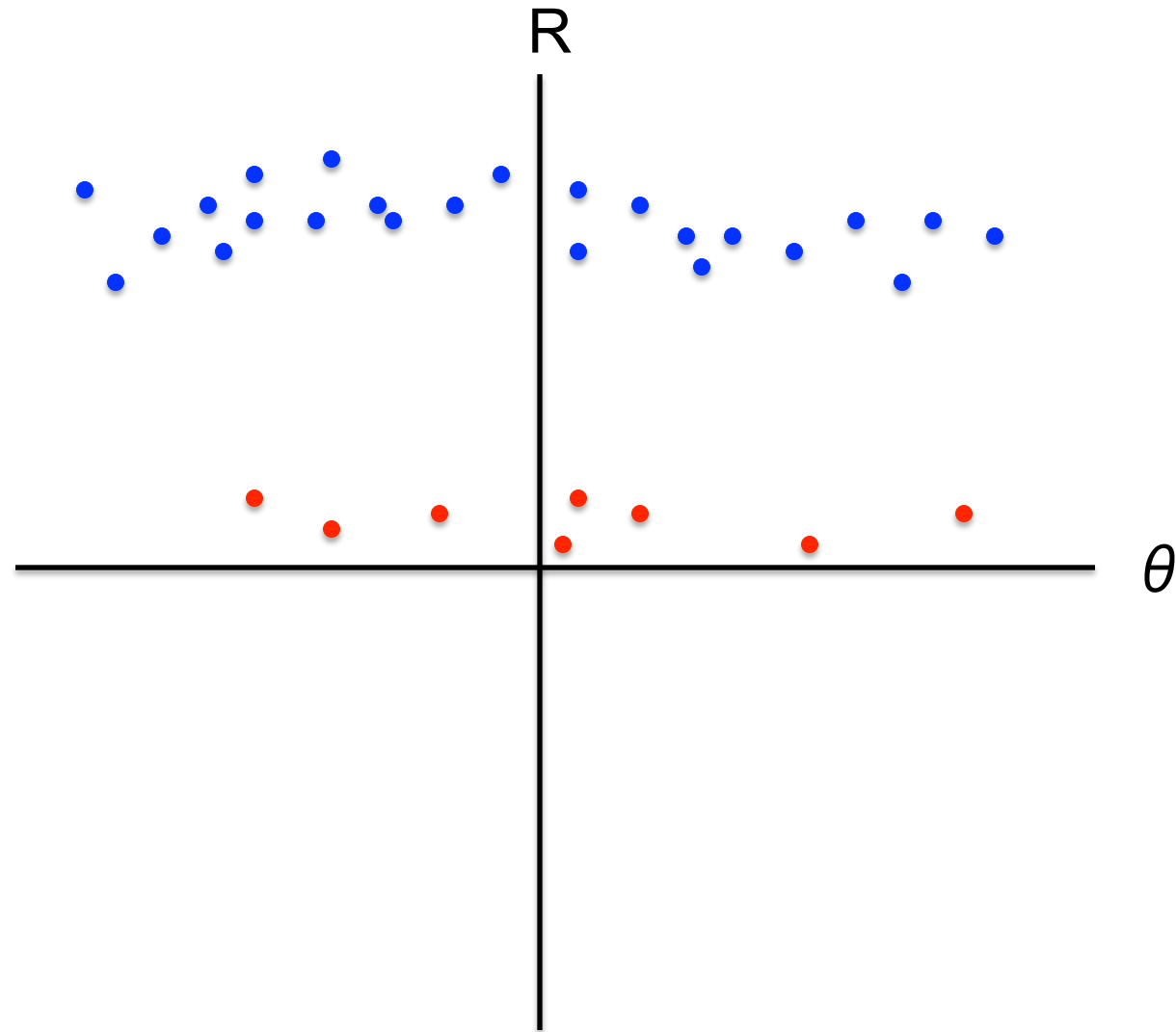


... and two clusters here

K-means not able to properly cluster



Changing the features (distance function)  
can help





# Fundamentals of AI

## Clustering

# Density-based and graph-based clustering\*

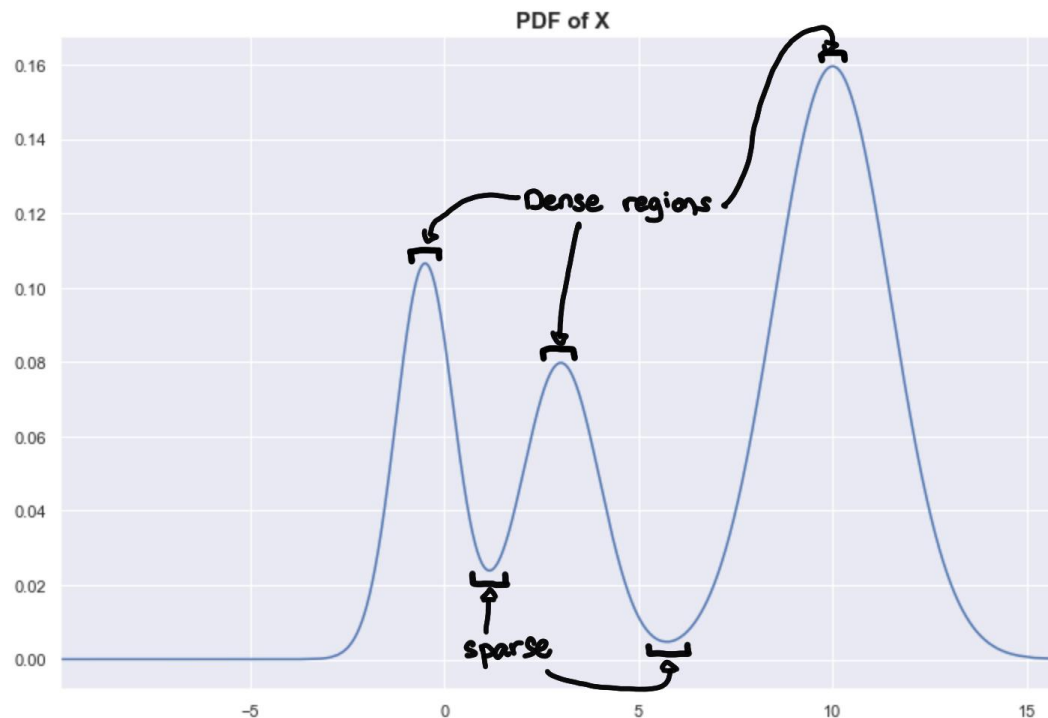
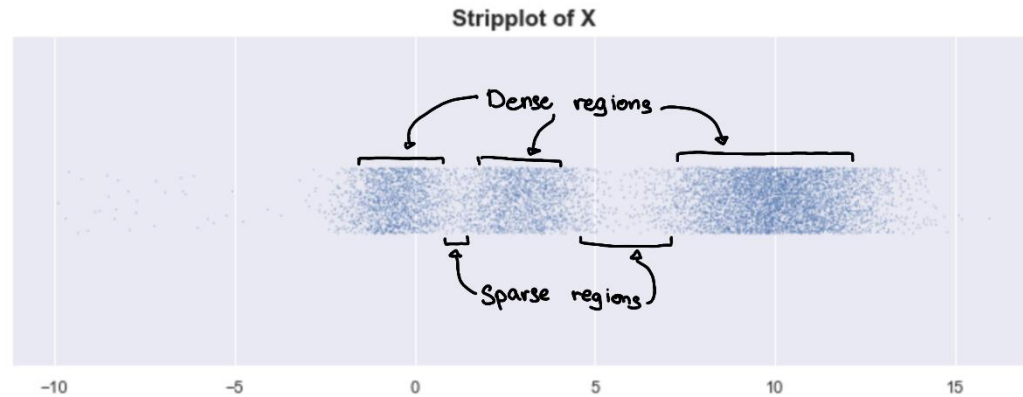
Some images in this lecture are used from: <https://www.kdnuggets.com/2020/02/understanding-density-based-clustering.html>

# Distance-based clustering and its limitations

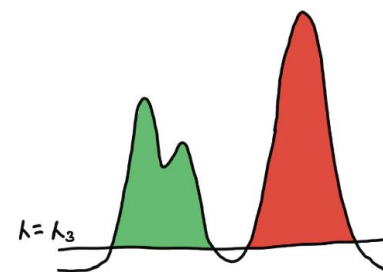
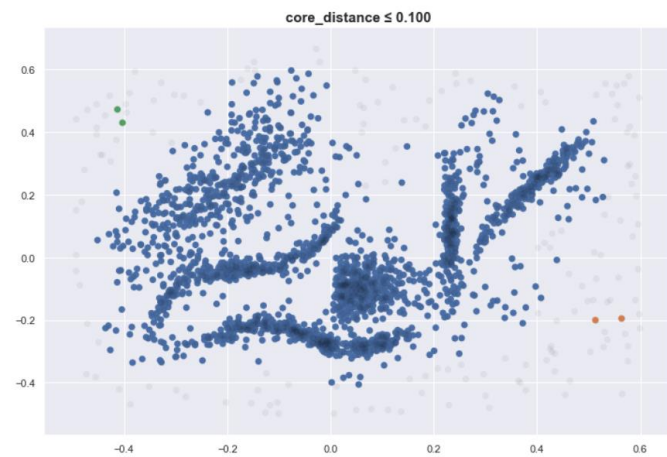
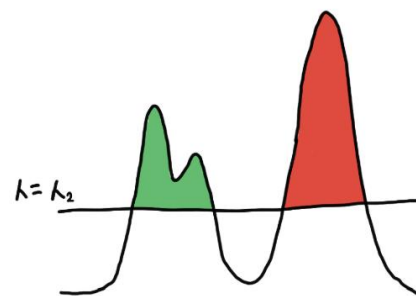
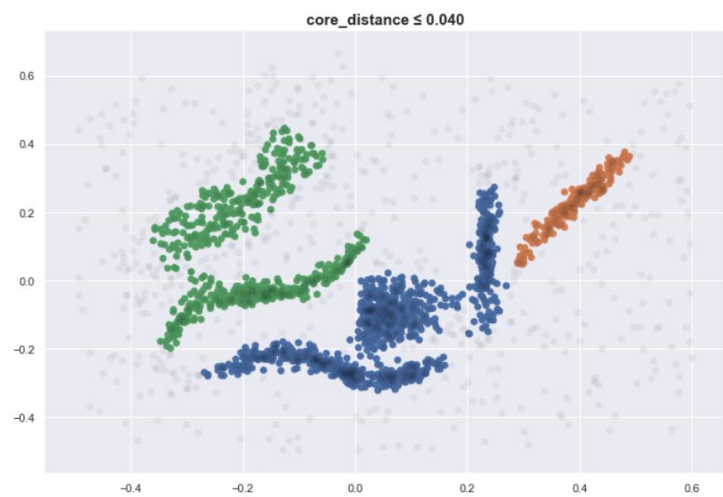
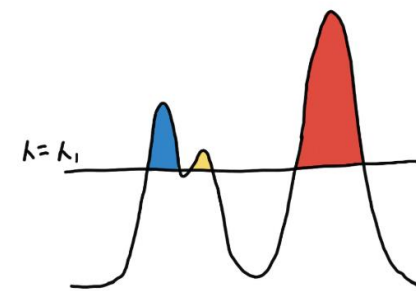
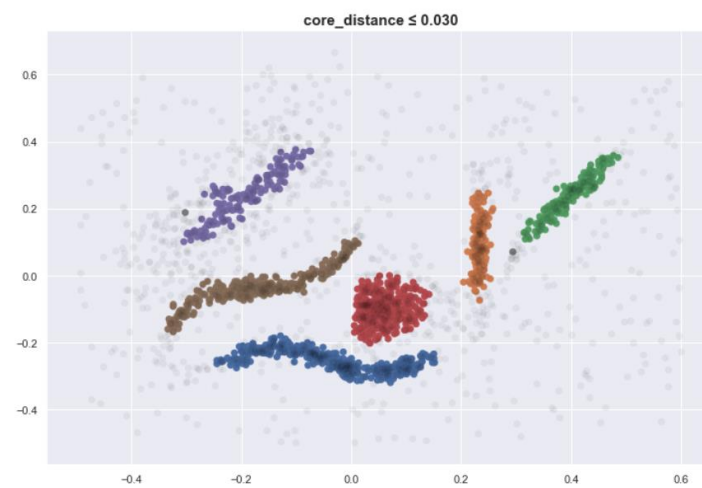
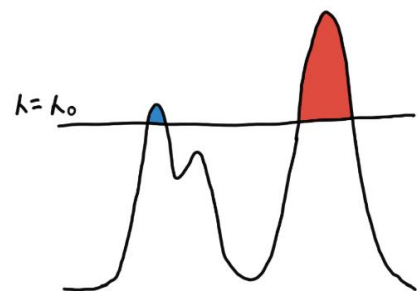
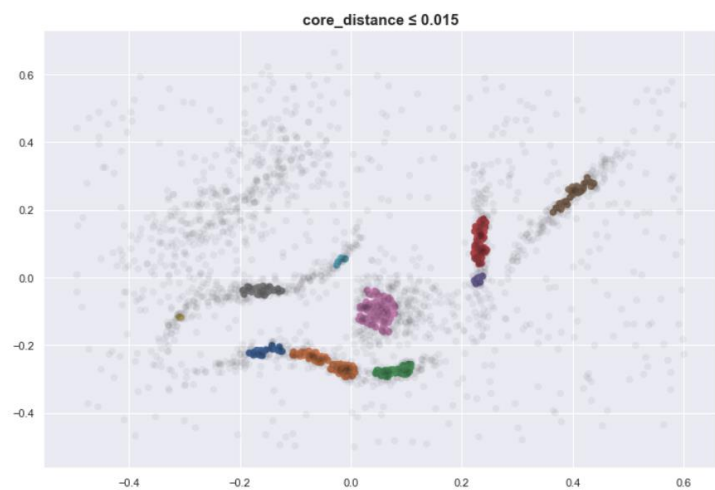
- Hard to find clusters with irregular shapes
- Hard to specify the number of clusters
- Some points are 'in between' clusters (outliers or background noise)



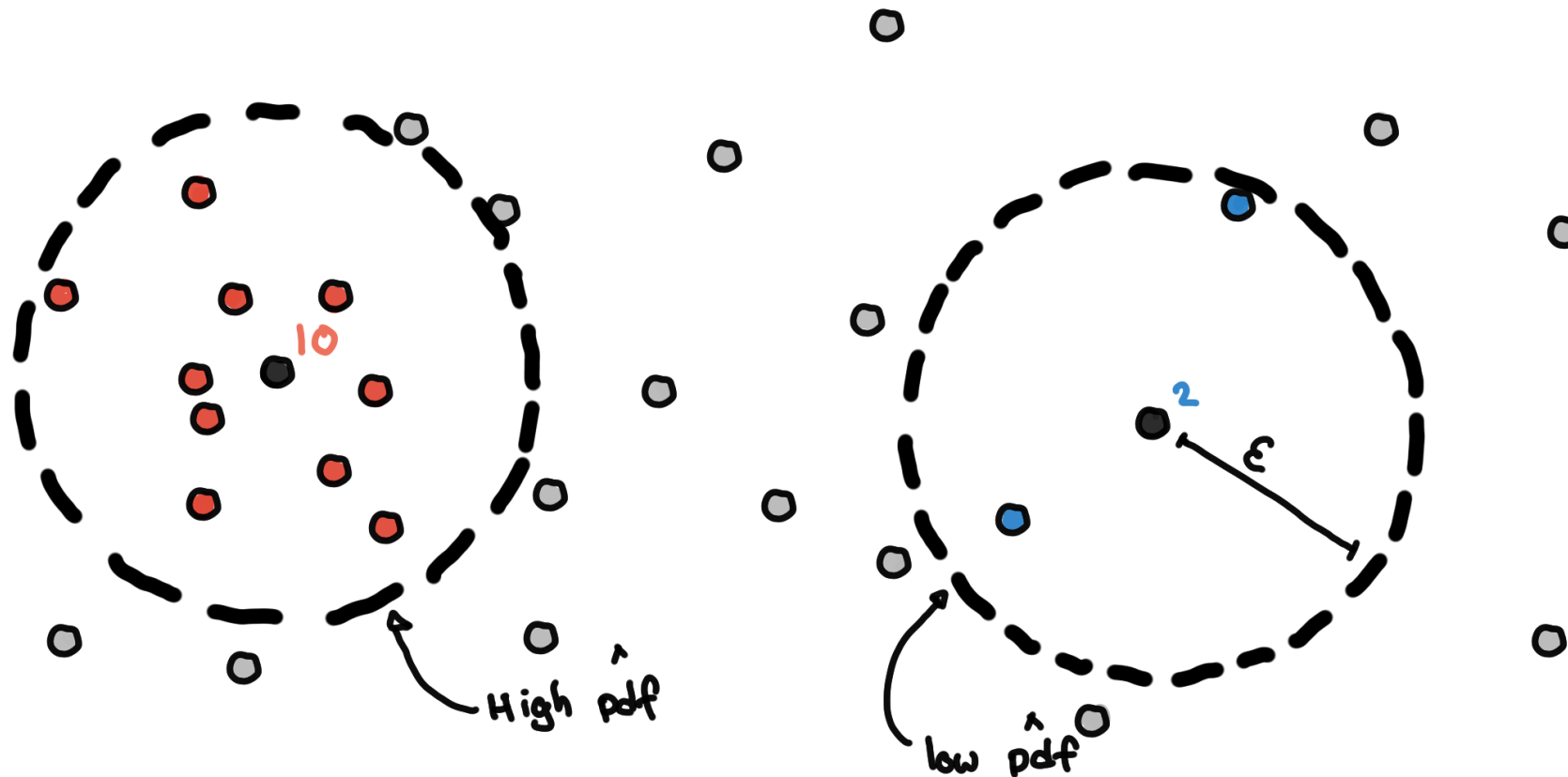
# New concept: cluster as a probability density peak



Cool, but how to define PDF in multi-dimensional space?  
Expensive and better to avoid at all



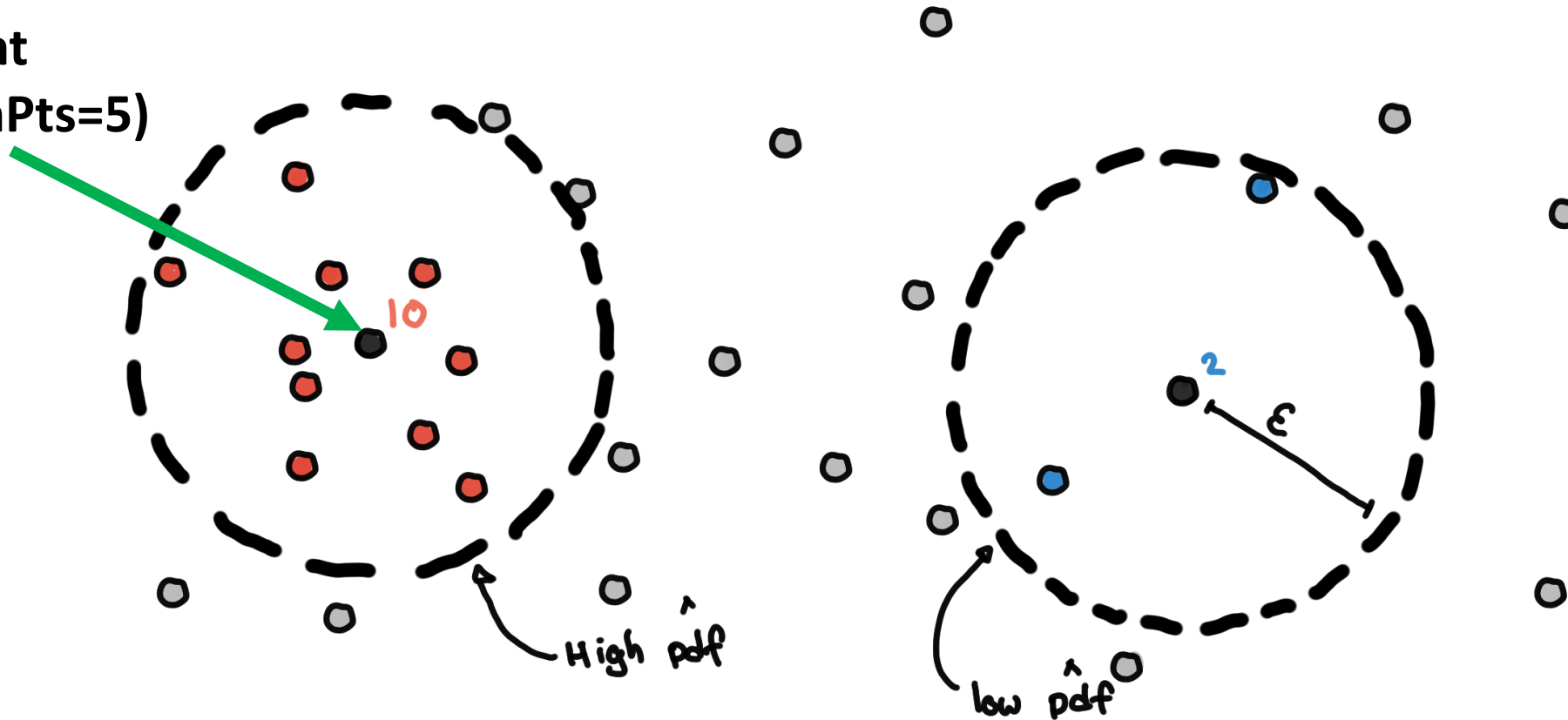
Trick: count neighbours within  $\epsilon$ -radius



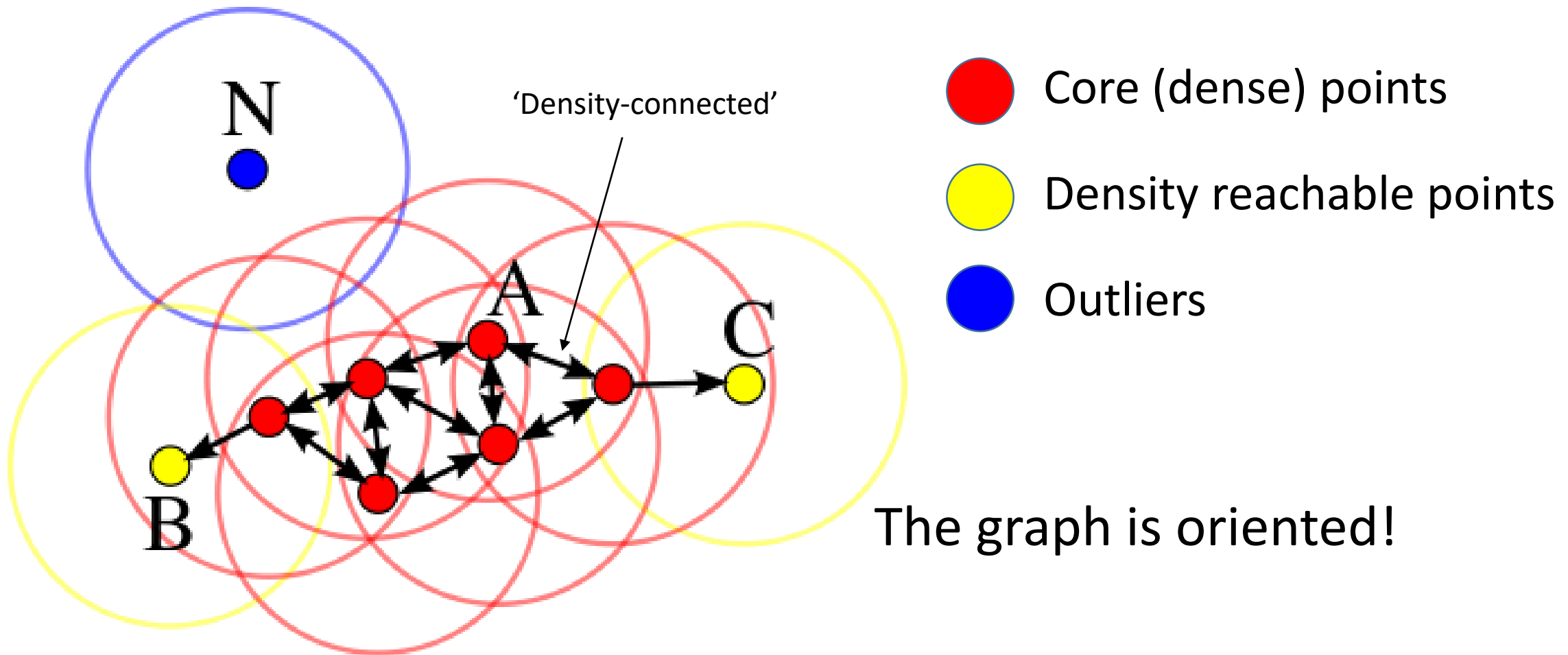
# DBSCAN (Ester et al, 1996)

Parameters:  $\epsilon$  and minPts

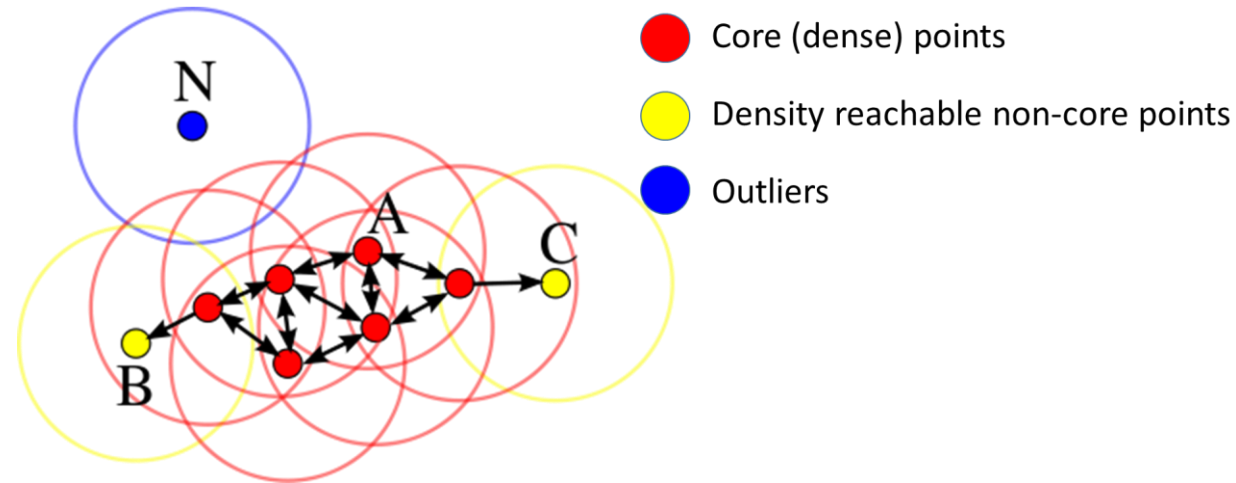
Core point  
(e.g., minPts=5)



# DBSCAN: graph of core points and density-reachable (peripheral) points



# DBSCAN: graph of core points and density-reachable (peripheral) points



A cluster then satisfies two properties:

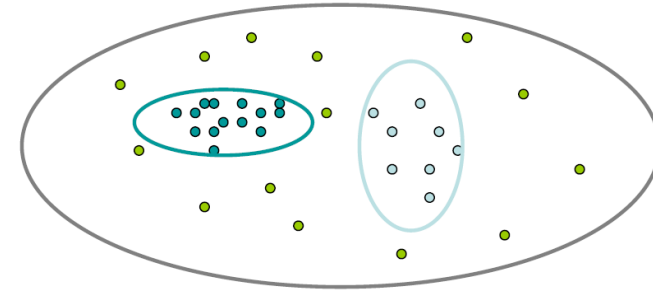
1. All points within the cluster are mutually density-connected.
2. If a point is density-reachable from some point of the cluster, it is part of the cluster as well.



# DBSCAN: the Algorithm

- Arbitrary select a point  $p$
- Retrieve all points density-reachable from  $p$  wrt  $Eps$  and  $MinPts$
- If  $p$  is a core point, a cluster is formed
- If  $p$  is a border point, no points are density-reachable from  $p$  and DBSCAN visits the next point of the database
- Continue the process until all of the points have been processed

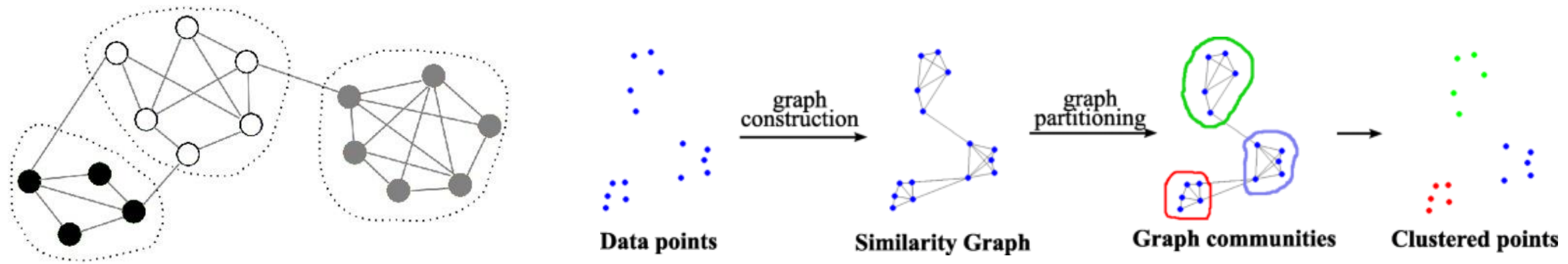
# Comments on DBSCAN



- Complexity is  $O(n \log n)$
- Unlike k-means and hierarchical, deal with the notion of noise
- Different clusters may have very different densities
- Very sensitive to the choice of  $\epsilon$
- Concentration of measures will spoil everything in high intrinsic dimensionalities
- Extensions: OPTICS, HDBSCAN, GDBSCAN
- Scikit learn implementation (arbitrary  $L_p$  metrics, accelerated neighbor search)

# Graph-based clustering algorithms

- Cluster = tight community of the KNN graph



- The quality of communities is determined by **modularity**

# Various definitions of the modularities

- E.g., **Louvain modularity**

$$Q = \frac{1}{2m} \sum_{ij} \left[ A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j),$$

In range [-0.5; 1]

where

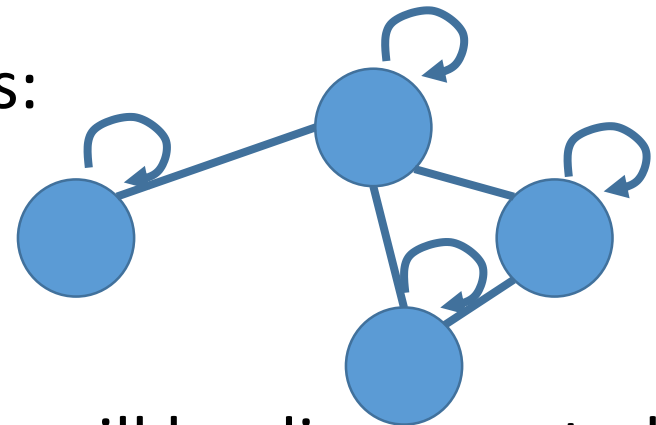
- $A_{ij}$  represents the edge weight between nodes  $i$  and  $j$ ;
- $k_i$  and  $k_j$  are the sum of the weights of the edges attached to nodes  $i$  and  $j$ , respectively;
- $m$  is the sum of all of the edge weights in the graph;
- $c_i$  and  $c_j$  are the communities of the nodes; and
- $\delta$  is **Kronecker delta function** ( $\delta(x, y) = 1$  if  $x = y$ , 0 otherwise).

# Louvain clustering algorithm (a greedy one)

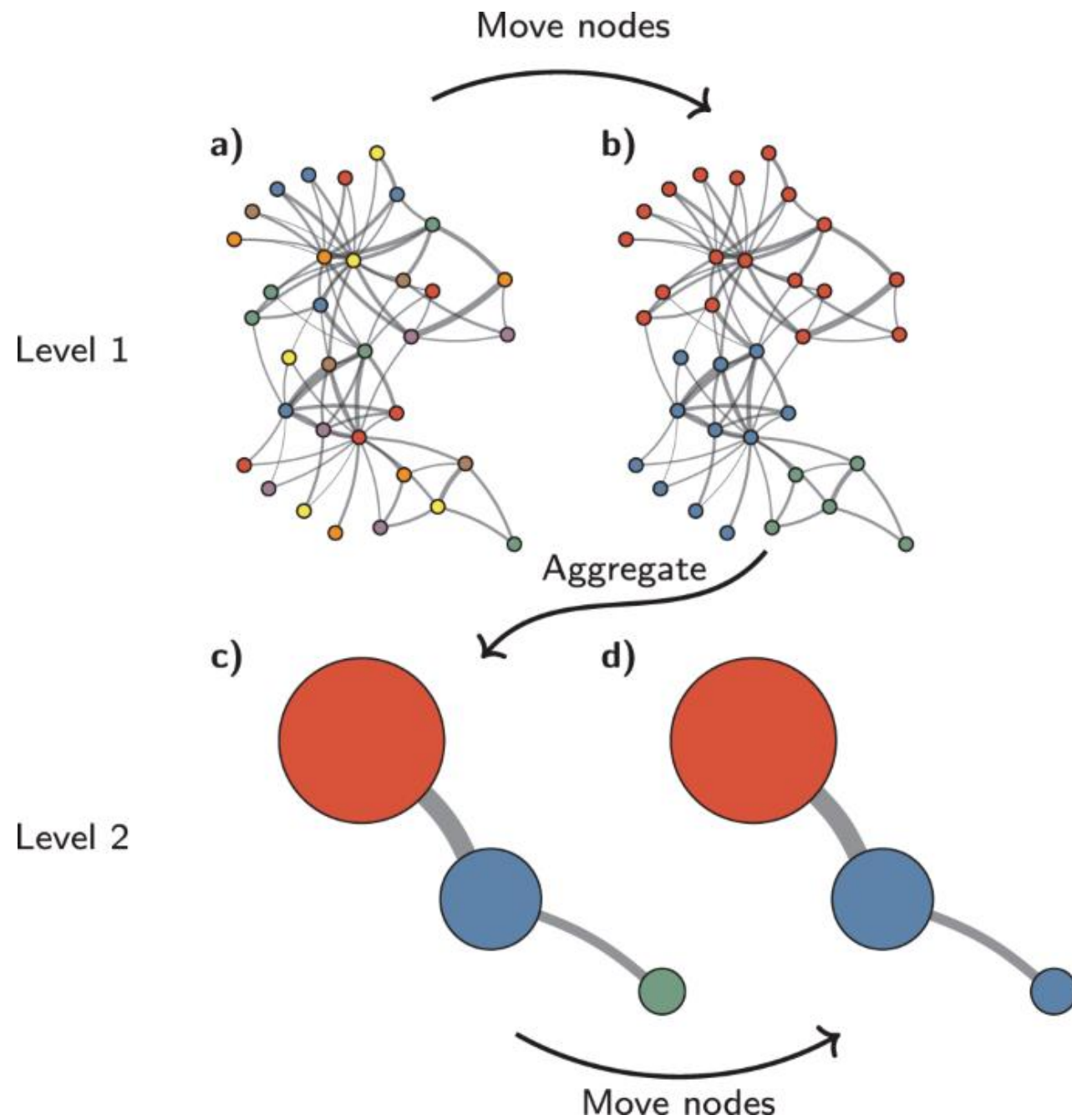
Base community search:

- At first, one node = one community
- We swap node  $i$  from its own community to the community of each of its neighbours
- For each such a swap, change in modularity is computed
- If no increase of modularity possible  $i$  remains in its own community

Finding communities in the graph of communities:



Cluster until one community remains or the graph will be disconnected



Graph-based  
clustering became  
new killer application  
in life sciences,  
replacing the  
hierarchical clustering



2 million data points – individual cells from mouse embryo



(from Cao et al, Nature, 2019)

# Spectral Clustering

A Primer to Graph-based Clusters

*Panthadeep Bhattacharjee*  
*CSE, NIT Rourkela*



# Spectral Clustering Algorithms

## ■ Three basic stages:

### ■ 1) Pre-processing

- Construct a matrix representation of the graph

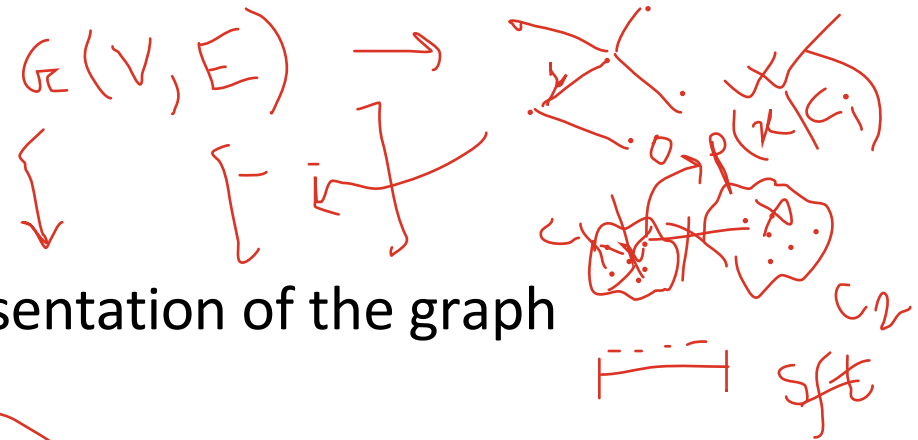
### ■ 2) Decomposition

- Compute eigenvalues and eigenvectors of the matrix
- Map each point to a lower-dimensional representation based on one or more eigenvectors

### ■ 3) Grouping

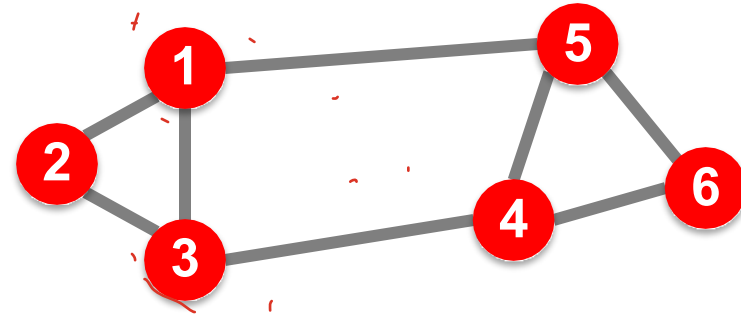
- Assign points to two or more clusters, based on the new representation

## ■ But first, let's define the problem



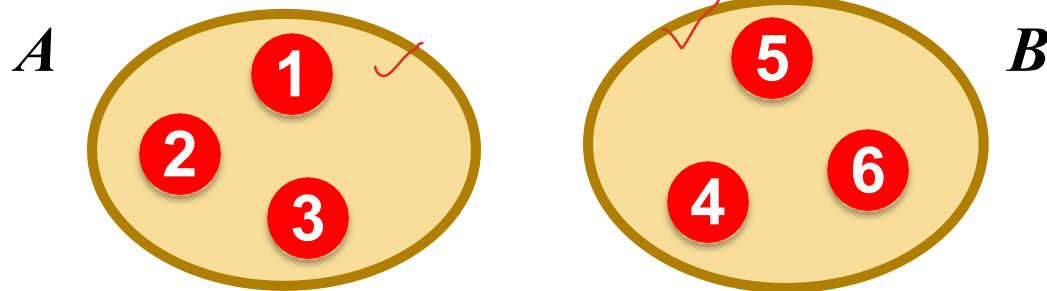
# Graph Partitioning

■ Undirected graph  $G(V, E)$ :



■ Bi-partitioning task:

■ Divide vertices into two disjoint groups  $A, B$



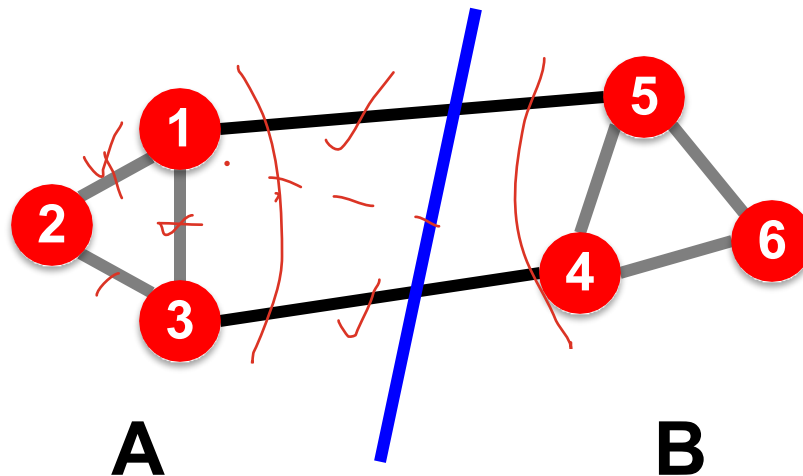
■ Questions:

- How can we define a “good” partition of  $G$ ?
- How can we efficiently identify such a partition?

# Graph Partitioning

## ■ What makes a good partition?

- ✓ ■ Maximize the number of within-group connections
- ✓ ■ Minimize the number of between-group connections



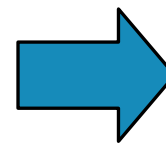
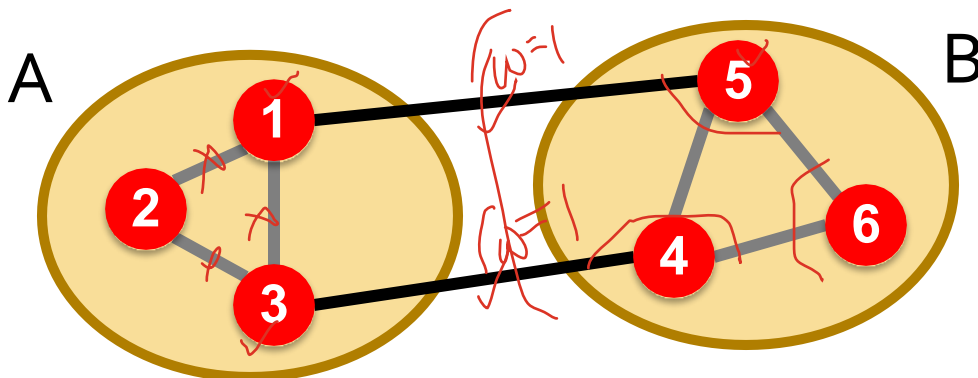
# Graph Cuts

- Express partitioning objectives as a function of the “edge cut” of the partition

✂️ **Cut:** Set of edges with one endpoint in each group:

$$\text{cut}(A, B) = \sum_{i \in A, j \in B} w_{ij}$$

If the graph is weighted  $w_{ij}$  is the weight, otherwise, all  $w_{ij} \in \{0, 1\}$



$$\text{cut}(A, B) = 2$$

✓

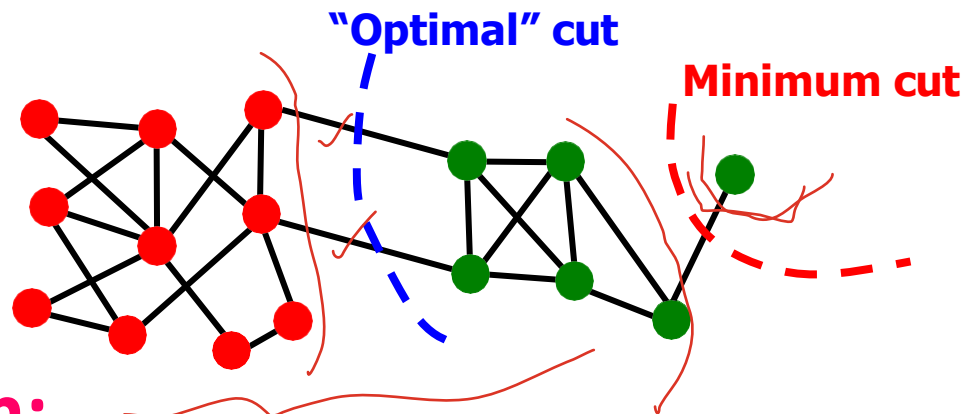
# Graph Cut Criterion

## ✓ Criterion: **Minimum-cut**

- Minimize weight of connections between groups

$$\arg \min_{A,B} \text{cut}(A,B)$$

## ✓ Degenerate case:



## ✓ Problem:

- Only considers external cluster connections
- Does not consider internal cluster connectivity

# Graph Cut Criterion

## ■ Criterion: **Conductance** [Shi-Malik, '97]

- Connectivity between groups relative to the density of each group

$$\phi(A, B) = \frac{\text{cut}(A, B)}{\min(\text{vol}(A), \text{vol}(B))}$$

$\text{vol}(A)$ : total weighted degree of the nodes in  $A$ :

$\text{vol}(A) = \sum_{i \in A} k_i$  (number of edge end points in  $A$ )

## ■ Why use this criterion?

- Produces more balanced partitions

## ■ How do we efficiently find a good partition?

- **Problem:** Computing best conductance cut is NP-hard

# Spectral Graph Partitioning

- $A$ : adjacency matrix of undirected  $G$ 
  - $A_{ij} = 1$  if  $(i, j)$  is an edge, else 0
- $x$  is a vector in  $\mathbb{R}^n$  with components  $(x_1, \dots, x_n)$ 
  - Think of it as a label/value of each node of  $G$
- What is the meaning of  $A \cdot x$ ?

$$\begin{bmatrix} a_{11} & \dots & a_{1n} \\ \vdots & & \vdots \\ a_{n1} & \dots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

$$y_i = \sum_{j=1}^n A_{ij} x_j = \sum_{(i,j) \in E} x_j$$

- Entry  $y_i$  is a sum of labels  $x_j$  of neighbors of  $i$

# What is the Meaning of $Ax$ ?

- $j^{\text{th}}$  coordinate of  $A \cdot x$  :
  - Sum of the  $x$ -values of neighbors of  $j$
  - Make this a new  $x$ -value at node  $j$

$$\begin{bmatrix} a_{11} & \dots & a_{1n} \\ \vdots & & \vdots \\ a_{n1} & \dots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \lambda \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$

$$\underline{A \cdot x = \lambda \cdot x}$$

$$\hookrightarrow \phi: V \rightarrow V$$

## ✓ Spectral Graph Theory:

- Analyze the “spectrum” of matrix representing  $G$
- **Spectrum:** Eigenvectors  $x^{(i)}$  of a graph, ordered by the magnitude (strength) of their corresponding eigenvalues  $\lambda_i$ :

$$\Lambda = \{\lambda_1, \lambda_2, \dots, \lambda_n\}$$

$$\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$$

Note: We sort  $\lambda_i$  in ascending (not descending) order!



# Example: $d$ -Regular Graph

- Suppose all nodes in  $G$  have degree  $d$  ( $G$  is  $d$ -regular) and  $G$  is connected
- **What are some eigenvalues/vectors of  $G$ ?**

$A \cdot x = \lambda \cdot x$  What is  $\lambda$ ? What  $x$ ?

- Let's try:  $x = (1, 1, \dots, 1)$  ✓
- Then:  $A \cdot x = (d, d, \dots, d) = \lambda \cdot x$ . So:  $\lambda = d$  ✓
- **We found an eigenpair of  $G$ :**  
 $x = (1, 1, \dots, 1)$ ,  $\lambda = d$  ✓

- $d$  is the largest eigenvalue of  $A$  (see next slide)

Remember the meaning of  $y = A \cdot x$ :

Note, this is just one eigenpair.

An  $n$  by  $n$  matrix can have up to  $n$  eigenpairs.

$$y_i = \sum_{j=1}^n A_{ij}x_j = \sum_{(i,j) \in E} x_j$$

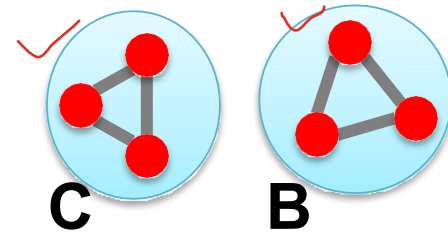
# $d$ is the Largest Eigenvalue of $A$

- $G$  is  $d$ -regular connected,  $A$  is its adjacency matrix
- **Claim:**
  - (1)  $d$  has multiplicity of 1 (there is only 1 eigenvector associated with eigenvalue  $d$ )
  - (2)  $d$  is the largest eigenvalue of  $A$
- **Proof:**
  - To obtain value eigval  $d$  we needed  $x_i = x_j$  for every  $i, j$
  - This means  $x = c \cdot (1, 1, \dots, 1)$  for some const.  $c$
  - **Define:** Set  $S$  = nodes  $i$  with maximum value of  $x_i$
  - Then consider some vector  $y$  which is not a multiple of vector  $(1, \dots, 1)$ . So not all nodes  $i$  (with labels  $y_i$ ) are in  $S$
  - Consider some node  $j \in S$  and a neighbor  $i \notin S$  then node  $j$  gets a value strictly less than  $d$
  - So  $y$  is not eigenvector! And so  $d$  is the largest eigenvalue!

# Example: Graph on 2 Components

## ■ What if $G$ is not connected?

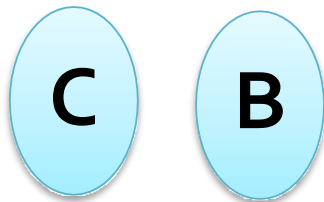
- $G$  has 2 components, each  $d$ -regular



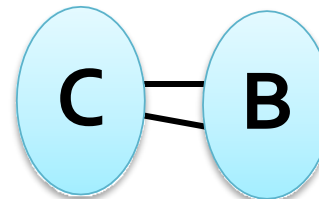
## ■ What are some eigenvectors?

- $x$  = Put all 1s on  $C$  and 0s on  $B$  or vice versa
  - $x' = (\underbrace{1, \dots, 1}_{|C|}, \underbrace{0, \dots, 0}_{|B|})^T$  then  $A \cdot x' = (d, \dots, d, 0, \dots, 0)^T$
  - $x'' = (\underbrace{0, \dots, 0}_{|C|}, \underbrace{1, \dots, 1}_{|B|})^T$  then  $A \cdot x'' = (0, \dots, 0, d, \dots, d)^T$
  - And so in both cases the corresponding  $\lambda = d$

## ■ A bit of intuition:



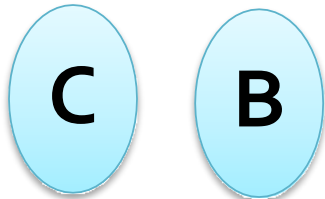
$$\lambda_n = \lambda_{n-1}$$



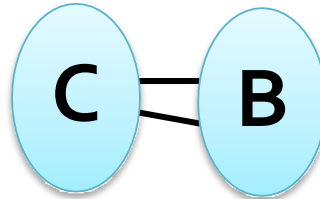
$$\lambda_n - \lambda_{n-1} \approx 0$$

2<sup>nd</sup> largest eigval.  
 $\lambda_{n-1}$  now has  
 value very close  
 to  $\lambda_n$

# More Intuition



$$\lambda_n = \lambda_{n-1}$$



$$\lambda_n - \lambda_{n-1} \approx 0$$

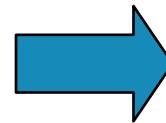
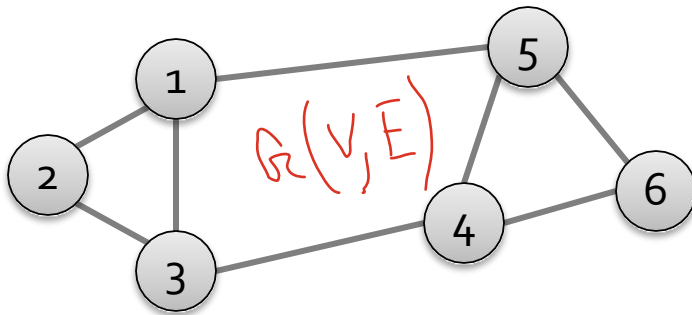
2<sup>nd</sup> largest eigval.  
 $\lambda_{n-1}$  now has  
value very close  
to  $\lambda_n$

- If the d-regular graph is connected (right example) then we already know that  $\mathbf{x}_n = (\mathbf{1}, \dots \mathbf{1})$  is an eigenvector
- Eigenvectors are orthogonal so then the components of  $\mathbf{x}_{n-1}$  must sum to  $\mathbf{0}$ 
  - Why?  $\mathbf{x}_n \cdot \mathbf{x}_{n-1} = \mathbf{0}$  then  $\sum_i \mathbf{x}_n[i] \cdot \mathbf{x}_{n-1}[i] = \sum_i \mathbf{x}_{n-1}[i] = \mathbf{0}$
  - $\mathbf{x}_{n-1}$  “splits” the nodes into two groups
    - $\mathbf{x}_{n-1}[i] > \mathbf{0}$  vs.  $\mathbf{x}_{n-1}[i] < \mathbf{0}$
  - So we in principle could look at the eigenvector of the 2<sup>nd</sup> largest eigenvalue and declare nodes with positive label in **C** and negative label in **B**. (but there are still many details for us to figure out here)

# Matrix Representations

## ✓ Adjacency matrix ( $A$ ):

- $n \times n$  matrix
- $A = [a_{ij}]$ ,  $a_{ij} = 1$  if edge between node  $i$  and  $j$



|   | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 0 | 1 | 0 |
| 2 | 1 | 0 | 1 | 0 | 0 | 0 |
| 3 | 1 | 1 | 0 | 1 | 0 | 0 |
| 4 | 0 | 0 | 1 | 0 | 1 | 1 |
| 5 | 1 | 0 | 0 | 1 | 0 | 1 |
| 6 | 0 | 0 | 0 | 1 | 1 | 0 |

## ■ Important properties:

- ✓ ■ Symmetric matrix
- Has  $n$  real eigenvalues
- Eigenvectors are real-valued and orthogonal

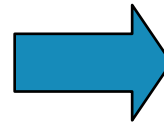
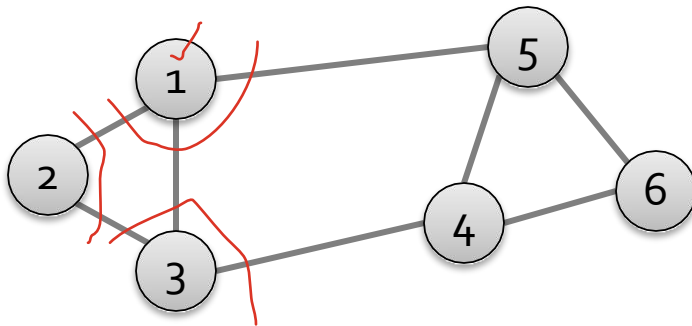
$$A = A^T$$

$$x_1, x_2$$

# Matrix Representations

## ■ Degree matrix (D):

- $n \times n$  diagonal matrix
- $D=[d_{ii}]$ ,  $d_{ii}$  = degree of node  $i$

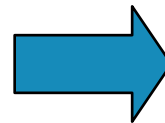
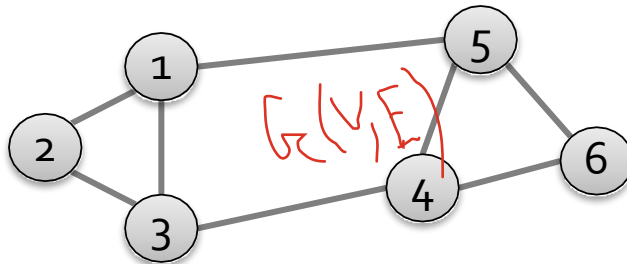


|   | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | 3 | 0 | 0 | 0 | 0 | 0 |
| 2 | 0 | 2 | 0 | 0 | 0 | 0 |
| 3 | 0 | 0 | 3 | 0 | 0 | 0 |
| 4 | 0 | 0 | 0 | 3 | 0 | 0 |
| 5 | 0 | 0 | 0 | 0 | 3 | 0 |
| 6 | 0 | 0 | 0 | 0 | 0 | 2 |

# Matrix Representations

## ✓ Laplacian matrix ( $L$ ):

- $n \times n$  symmetric matrix



|   | 1  | 2  | 3  | 4  | 5  | 6  |
|---|----|----|----|----|----|----|
| 1 | 3  | -1 | -1 | 0  | -1 | 0  |
| 2 | -1 | 2  | -1 | 0  | 0  | 0  |
| 3 | -1 | -1 | 3  | -1 | 0  | 0  |
| 4 | 0  | 0  | -1 | 3  | -1 | -1 |
| 5 | -1 | 0  | 0  | -1 | 3  | -1 |
| 6 | 0  | 0  | 0  | -1 | -1 | 2  |

$$L = D - A$$

## ■ What is trivial eigenpair?

- ✓  $x = (1, \dots, 1)$  then  $L \cdot x = 0$  and so  $\lambda = \lambda_1 = 0$

## ■ Important properties of $L$ :

- ✓ Eigenvalues are non-negative real numbers
- Eigenvectors are real (and always orthogonal)

# 3 Facts About the Laplacian $L$

✓ (a) All eigenvalues are  $\geq 0$  ✓

(b)  $x^T L x = \sum_{ij} L_{ij} x_i x_j \geq 0$  for every  $x$  ✓

(c)  $L$  can be written as  $\underline{L} = N^T \cdot N$

■ That is,  $L$  is positive semi-definite

■ **Proof: (the 3 facts are saying the same thing)**

■ (c)  $\Rightarrow$  (b):  $x^T L x = x^T N^T N x = (N x)^T (N x) \geq 0$

■ As it is just the square of length of  $N x$

■ (b)  $\Rightarrow$  (a): Let  $\lambda$  be an eigenvalue of  $L$ . Then by (b)  $x^T L x \geq 0$  so  $x^T L x = x^T \lambda x = \lambda x^T x \Rightarrow \lambda \geq 0$

■ (a)  $\Rightarrow$  (c): is also easy! Do it yourself.

$[x \cdot x]$   
 $x^T A x \geq 0$   
 $x \in \{0\}$



# $\lambda_2$ as an Optimization Problem

- **Fact: For symmetric matrix  $M$ :**

$$\underline{\lambda_2} = \min_{x : x^T w_1 = 0} \left( \frac{x^T M x}{x^T x} \right)$$

( $w_1$  is eigenvector corresponding to  $\lambda_1$ )

See next slide  
for the proof.  
Deriving this is  
a HW problem.

- **What is the meaning of  $\min x^T L x$  on  $G$ ?**

$$\begin{aligned} \blacksquare x^T L x &= \sum_{i,j;1}^n L_{ij} x_i x_j = \sum_{i,j;1}^n (D_{ij} - A_{ij}) x_i x_j \\ \blacksquare &= \sum_i D_{ii} x_i^2 - \sum_{(i,j) \in E} 2x_i x_j \\ \blacksquare &= \sum_{(i,j) \in E} (\underbrace{x_i^2 + x_j^2}_{\text{green brace}} - 2x_i x_j) = \sum_{(i,j) \in E} (\underbrace{x_i - x_j}_{\text{red brace}})^2 \end{aligned}$$

Node  $i$  has degree  $d_i$ . So, value  $x_i^2$  needs to be summed up  $d_i$  times.  
But each edge  $(i,j)$  has two endpoints so we need  $x_i^2 + x_j^2$

# Proof:

$$\lambda_2 = \min_{x : x^T w_1 = 0} \frac{x^T M x}{x^T x}$$

Details!

- Write  $x$  in basis of eigenvecs  $w_1, w_2, \dots, w_n$  of  $M$  and  $\lambda_i$  are corresponding eigenvalues. So,  $x = \sum_i^n \alpha_i w_i$
- Then we get:  $Mx = \sum_i \alpha_i \underbrace{Mw_i}_{\lambda_i w_i} = \sum_i \alpha_i \lambda_i w_i$
- So, what is  $x^T M x$ ?

$$\begin{aligned} x^T M x &= \left( \sum_i \alpha_i w_i \right)^T \left( \sum_i \alpha_i \lambda_i w_i \right) = \sum_{ij} \alpha_i \lambda_j \alpha_j \\ &= \sum_i \alpha_i^2 \lambda_i \underbrace{w_i^T w_i}_{=0 \text{ if } i \neq j, 1 \text{ otherwise}} = \sum_i \lambda_i \alpha_i^2 \end{aligned}$$

- Want minimize this over all unit vectors  $w$ :

$w = \min$  over choices of  $(\alpha_1, \dots, \alpha_n)$  so that:

- $x^k w_1 = 0$ , rewrite it as  $(\sum_i \alpha_i w_i) \cdot w_1 = 0$  and remember that  $w_i^T w_j = 0$  (because  $w$  are eigenvectors). Then  $\alpha_1 = 0$
- $\sum \alpha_i^2 = 1$  (unit length)

- So, to minimize this, set  $\alpha_2 = 1$  and the rest to 0  $\sum_i \lambda_i \alpha_i^2 = \lambda_2$

# Finding $x$ that Solves

$$\lambda_2 = \min_{x : x^T w_1 = 0} \frac{x^T M x}{x^T x}$$

## ■ What else do we know about $x$ ?

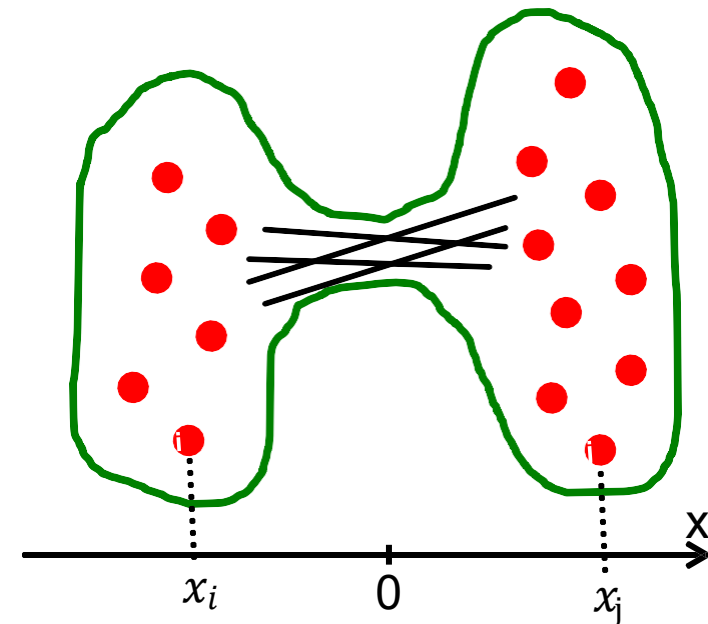
- $x$  is unit vector:  $\sum_i x_i^2 = 1$
- $x$  is orthogonal to  $1^{\text{st}}$  eigenvector  $(1, \dots, 1)$  thus:  
 $\sum_i x_i \cdot 1 = \sum_i x_i = 0$

## ■ Remember:

$$\lambda_2 = \min_{\substack{\text{All labelings} \\ \text{of nodes } i \text{ so} \\ \text{that } \sum x_i = 0}} \frac{\sum_{(i,j) \in E} (x_i - x_j)^2}{\sum_i x_i^2}$$

We want to assign values  $x_i$  to nodes  $i$  such that few edges cross 0.

(we want  $x_i$  and  $x_j$  to subtract each other)



# Find Optimal Cut [Fiedler'73]

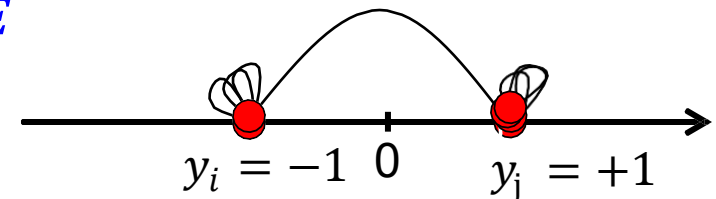
- Back to finding the optimal cut
- Express partition (A,B) as a vector

$$y_i = \begin{cases} +1 & \text{if } i \in A \\ -1 & \text{if } i \in B \end{cases}$$

- Enforce that  $|A| = |B| \rightarrow \sum_i y_i = 0$ 
  - Equivalent to being orthogonal to the trivial eigenvector  $(1, \dots, 1)$
- We can minimize the cut of the partition by finding a vector  $y$  that **minimizes**:

$$\arg \min_{y \in \{-1, +1\}^n} f(y) = \sum_{(i,j) \in E} (y_i - y_j)^2$$

Can't solve exactly. Let's relax  $y$   
and allow it to take any real value.

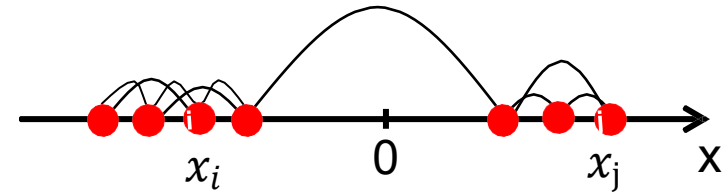


# Rayleigh Theorem

$$\lambda_2 = \min_{x : x^T w_1 = 0} \frac{x^T M x}{x^T x}$$

Slide 18

$$\min_{y \in \mathbb{R}^n : \sum_i y_i = 0, \sum_i y_i^2 = 1} f(y) = \min_{(i,j) \in E} (y_i - y_j)^2 = y^T L y$$



- **$\lambda_2 = \min_y f(y)$** : The minimum value of  $f(y)$  is given by the 2<sup>nd</sup> smallest eigenvalue  $\lambda_2$  of the Laplacian matrix  $L$
- **$x = \arg \min_y f(y)$** : The optimal solution for  $y$  is given by the eigenvector  $x$  corresponding to  $\lambda_2$ , referred to as the **Fiedler vector**
- Can use sign of  $x_i$  to determine cluster assignment

# Approx. Guarantee of Spectral

- Suppose there is a partition of  $G$  into  $A$  and  $B$  where  $|A| \leq |B|$ , s.t. “conductance” of the cut  $(A,B)$  is  $\beta = \frac{(\# \text{ edges from } A \text{ to } B)}{|A|}$  then  $\lambda_2 \leq 2\beta$ 

Note:  $|A| < |B|$
- This is the approximation guarantee of the spectral clustering: Spectral finds a cut that has at most **twice the conductance** as the optimal one of conductance  $\beta$ .
- **Proof:**
  - Let:  $a = |A|$ ,  $b = |B|$  and  $e = \#$  edges from  $A$  to  $B$
  - Enough to choose some  $x_i$  based on  $A$  and  $B$  such that:

$$\lambda_2 \leq \underbrace{\frac{\sum (x_i - x_j)^2}{\sum_i x_i^2}} \leq 2\beta \text{ (while also } \sum_i x_i = 0)$$

# Approx. Guarantee of Spectral

## ■ Proof (continued):

■ **1)** Let's set:  $x_i = \begin{cases} -\frac{1}{a} & \text{if } i \in A \\ +\frac{1}{b} & \text{if } i \in B \end{cases}$

Note:  $|A| < |B|$

■ Let's quickly verify that  $\sum_i x_i = 0$ :  $a \left(-\frac{1}{a}\right) + b \left(\frac{1}{b}\right) = 0$

■ **2)** Then:  $\frac{\sum (x_i - x_j)^2}{\sum_i x_i^2} = \frac{\sum_{i \in A, j \in B} \left(\frac{1}{b} + \frac{1}{a}\right)^2}{a \left(-\frac{1}{a}\right)^2 + b \left(\frac{1}{b}\right)^2} = \frac{e \cdot \left(\frac{1}{a} + \frac{1}{b}\right)^2}{\frac{1}{a} + \frac{1}{b}} =$

$$e \left(\frac{1}{a} + \frac{1}{b}\right) \leq e \left(\frac{1}{a} + \frac{1}{a}\right) = e \frac{2}{a} \leq 2\beta$$

$e$  ... number of edges between A and B

Which proves that the cost achieved by spectral is better than twice the OPT cost

# Approx. Guarantee of Spectral

- Putting it all together: The Cheeger inequality

$$\frac{\beta^2}{2k_{max}} \leq \lambda_2 \leq 2\beta$$

- where  $k_{max}$  is the maximum node degree in the graph
  - Note we only provide the 1<sup>st</sup> part:  $\lambda_2 \leq 2\beta$
  - We did not prove  $\frac{\beta^2}{2k_{max}} \leq \lambda_2$
- Overall this always certifies that  $\lambda_2$  always gives a useful bound



# So far...

- **How to define a “good” partition of a graph?**
  - Minimize a given graph cut criterion
- **How to efficiently identify such a partition?**
  - Approximate using information provided by the eigenvalues and eigenvectors of a graph
- **Spectral Clustering**

# Spectral Clustering Algorithm

## ■ Three basic stages:

### ✓ ■ 1) Pre-processing

- Construct a matrix representation of the graph

### ✓ ■ 2) Decomposition

- Compute eigenvalues and eigenvectors of the matrix
- ✓ ■ Map each point to a lower-dimensional representation based on one or more eigenvectors

### ✓ ■ 3) Grouping

- Assign points to two or more clusters, based on the new representation

$Ax = \lambda x$

$n \times n$

$n \times K$

# Spectral Partitioning Algorithm

## 1) Pre-processing:

- Build Laplacian matrix  $L$  of the graph



$$L = D - A$$

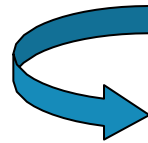


|   | 1  | 2  | 3  | 4  | 5  | 6  |
|---|----|----|----|----|----|----|
| 1 | 3  | -1 | -1 | 0  | -1 | 0  |
| 2 | -1 | 2  | -1 | 0  | 0  | 0  |
| 3 | -1 | -1 | 3  | -1 | 0  | 0  |
| 4 | 0  | 0  | -1 | 3  | -1 | -1 |
| 5 | -1 | 0  | 0  | -1 | 3  | -1 |
| 6 | 0  | 0  | 0  | -1 | -1 | 2  |

## 2)

## Decomposition:

- Find eigenvalues  $\lambda$  and eigenvectors  $x$  of the matrix  $L$
- Map vertices to corresponding components of  $x_2$

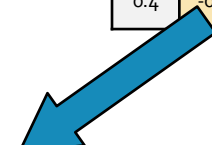


|     |
|-----|
| 0.0 |
| 1.0 |
| 3.0 |
| 3.0 |
| 4.0 |
| 5.0 |

|   |      |
|---|------|
| 1 | 0.3  |
| 2 | 0.6  |
| 3 | 0.3  |
| 4 | -0.3 |
| 5 | -0.3 |
| 6 | -0.6 |

$X =$

|     |      |      |      |      |      |
|-----|------|------|------|------|------|
| 0.4 | 0.3  | -0.5 | -0.2 | -0.4 | -0.5 |
| 0.4 | 0.6  | 0.4  | -0.4 | 0.4  | 0.0  |
| 0.4 | 0.3  | 0.1  | 0.6  | -0.4 | 0.5  |
| 0.4 | -0.3 | 0.1  | 0.6  | 0.4  | -0.5 |
| 0.4 | -0.3 | -0.5 | -0.2 | 0.4  | 0.5  |
| 0.4 | -0.6 | 0.4  | -0.4 | -0.4 | 0.0  |



How do we now find the clusters?

# Spectral Partitioning

## ■ 3) Grouping:

- Sort components of reduced 1-dimensional vector
- Identify clusters by splitting the sorted vector in two

## ■ How to choose a splitting point?

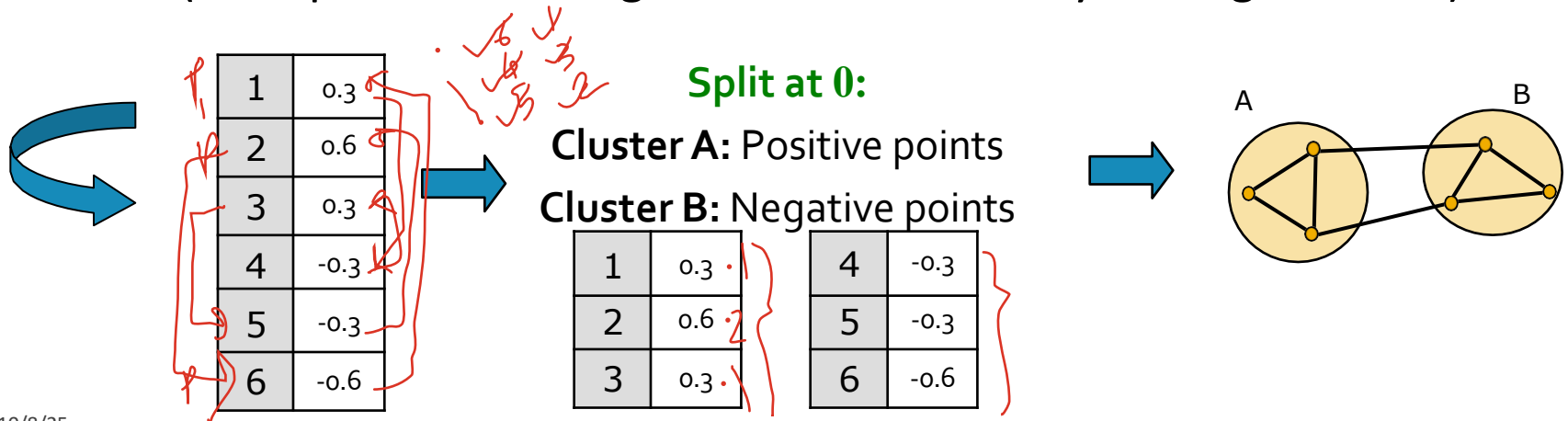
$x_2 \approx 2$

### ■ Naïve approaches:

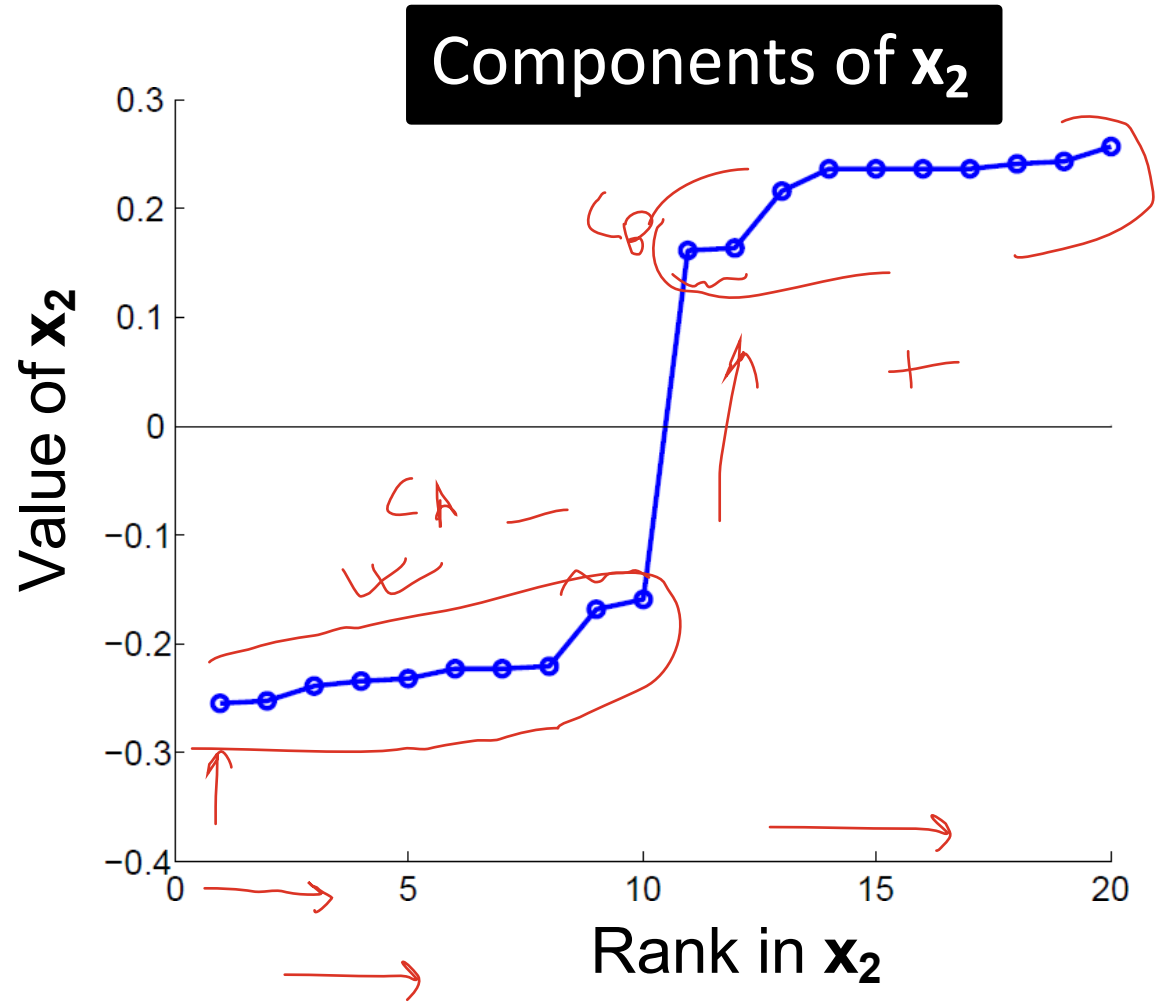
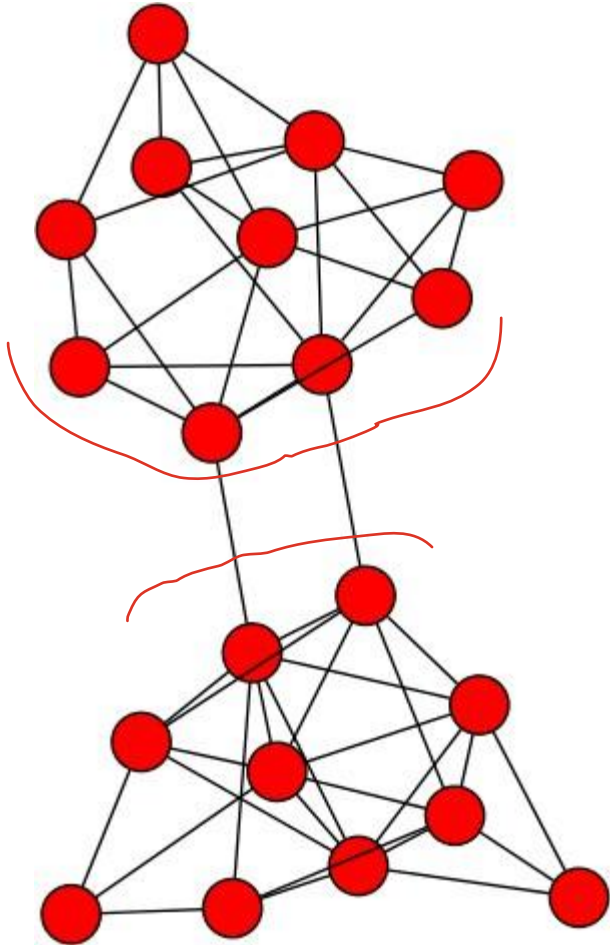
- Split at 0 or median value

### ■ More expensive approaches:

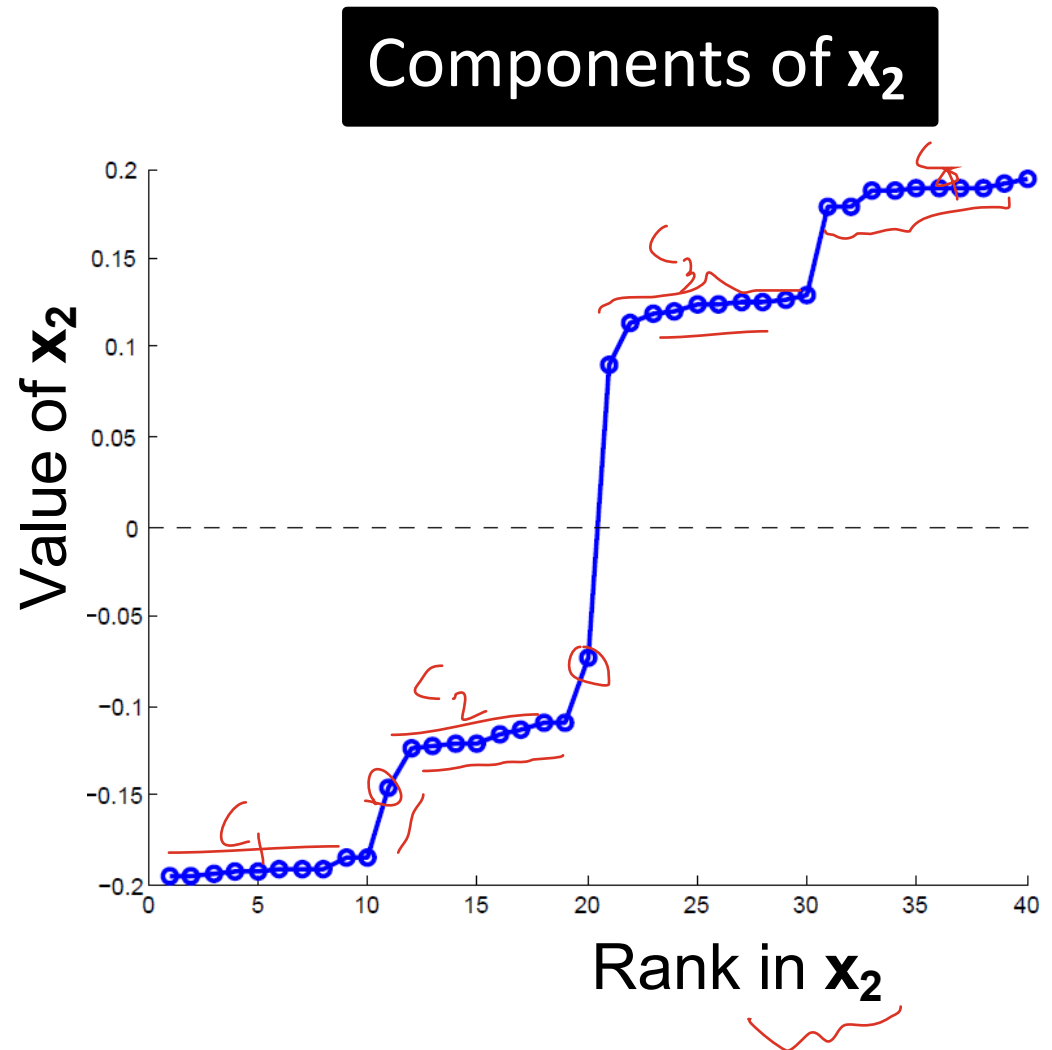
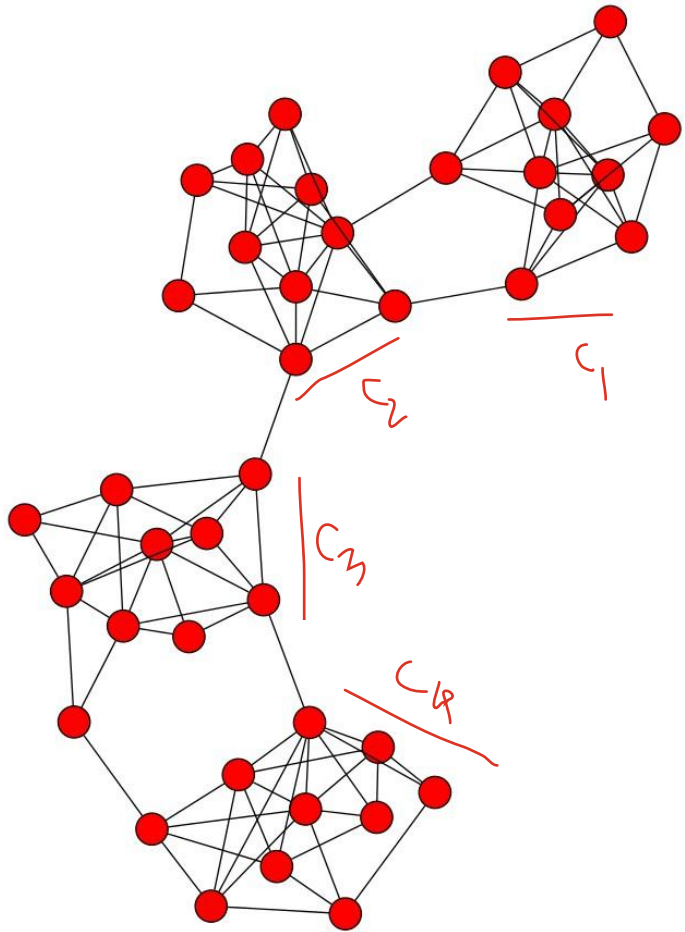
- Attempt to minimize normalized cut in 1-dimension (sweep over ordering of nodes induced by the eigenvector)



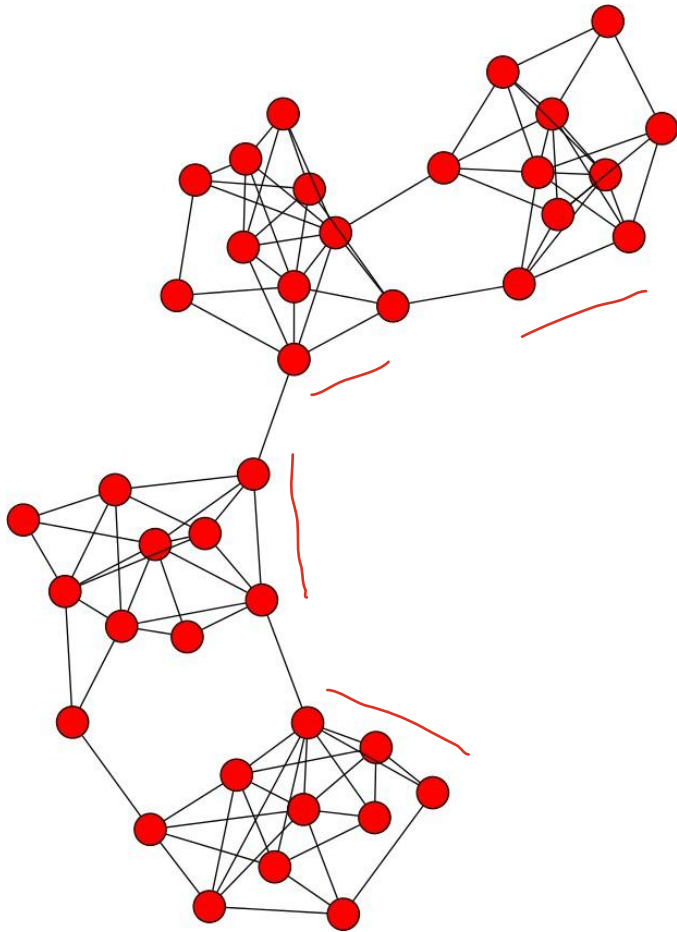
# Example: Spectral Partitioning



# Example: Spectral Partitioning

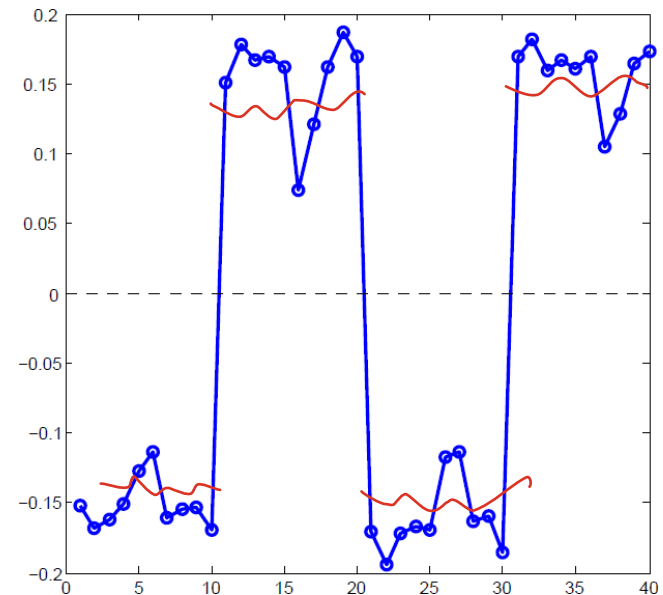
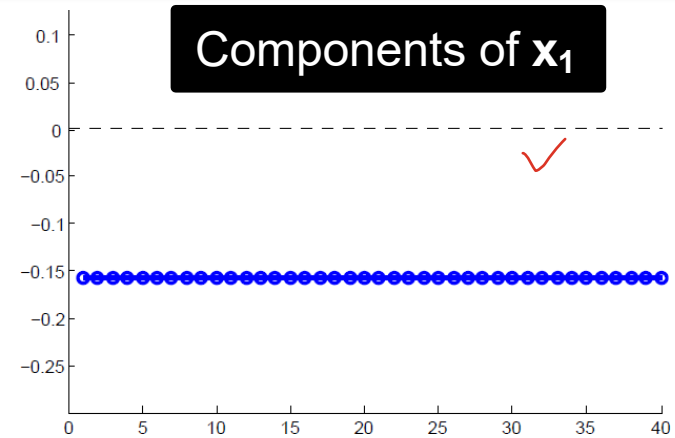


# Example: Spectral Partitioning



?

?



Components of  $x_3$

$> k_2$

# k-Way Spectral Clustering

- How do we partition a graph into  $k$  clusters?
- Two basic approaches:
  - **Recursive bi-partitioning** [Hagen et al., '92]
    - Recursively apply bi-partitioning algorithm in a hierarchical divisive manner
    - Disadvantages: Inefficient, unstable
  - **Cluster multiple eigenvectors** [Shi-Malik, '00]
    - Build a reduced space from multiple eigenvectors
      - Each node is now represented by  $k$  numbers
      - We then cluster (apply k-means) the nodes based on their  $k$ -dim representation
    - Commonly used in recent papers
    - A preferable approach...



# Why Use Multiple Eigenvectors?

- **Approximates the optimal cut** [Shi-Malik, '00]
  - Can be used to approximate optimal  $k$ -way normalized cut
- **Emphasizes cohesive clusters**
  - Increases the unevenness in the distribution of the data
  - Associations between similar points are amplified, associations between dissimilar points are attenuated
  - The data begins to “approximate a clustering”
- **Well-separated space**
  - Transforms data to a new “embedded space”, consisting of  $k$  orthogonal basis vectors
- Multiple eigenvectors prevent instability due to information loss

# How to Select k?

- **Eigengap:**

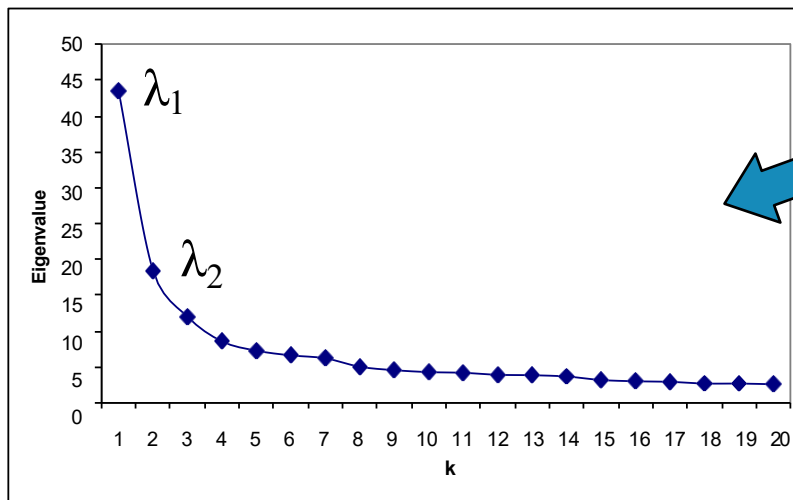
- The difference between two consecutive eigenvalues

- **Most stable clustering is generally given by the value  $k$  that maximizes eigengap  $\Delta_k$ :**

$$\Delta_k = |\lambda_k - \lambda_{k-1}|$$

Note eigenvalues are sorted in descending order

- **Example:**



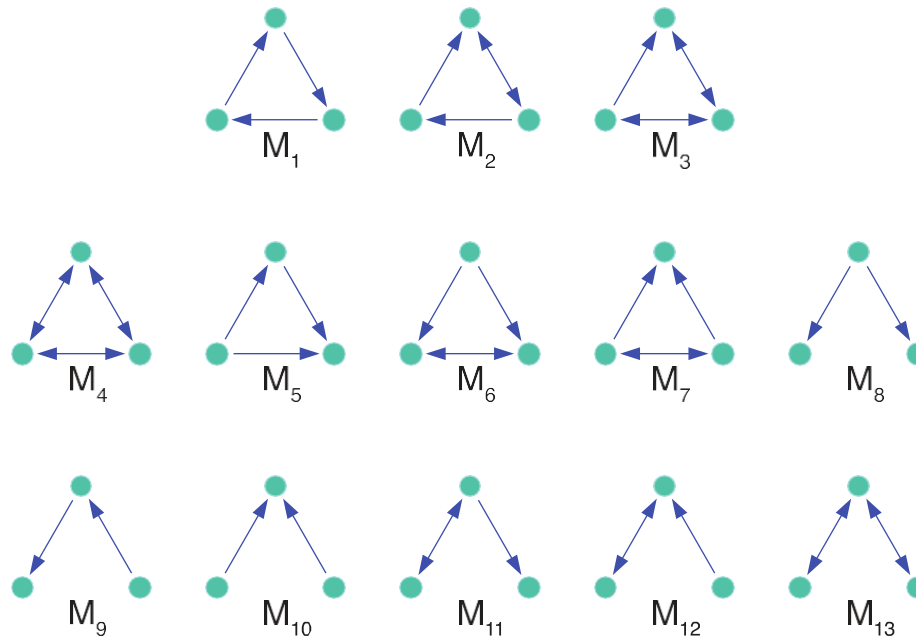
$$\max \Delta_k = |\lambda_2 - \lambda_1|$$

⇒ **Choose**  
 **$k = 2$**

# Motif-Based Spectral Clustering

# Motif-based spectral clustering

- What if we want our clustering based on other patterns (not edges)?

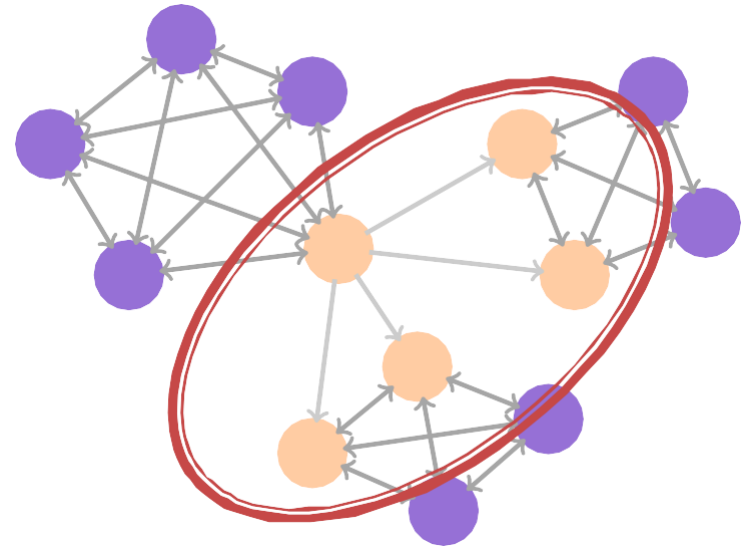
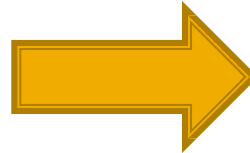
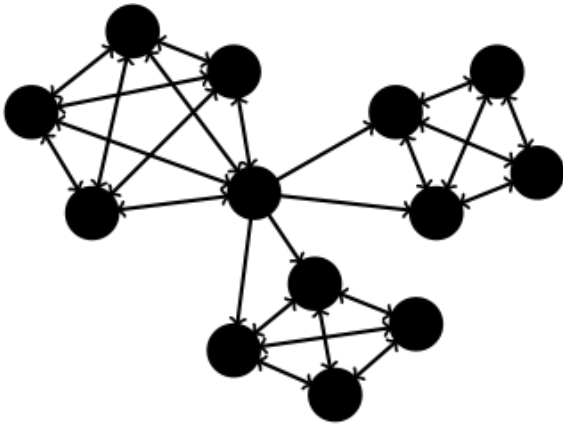


Small subgraphs (motifs, graphlets) are building blocks of networks [Milo et al., '02]

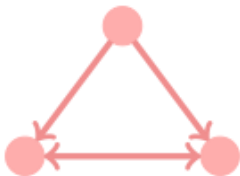
# Topic 1: Modules of Motifs

Find modules based on motifs!

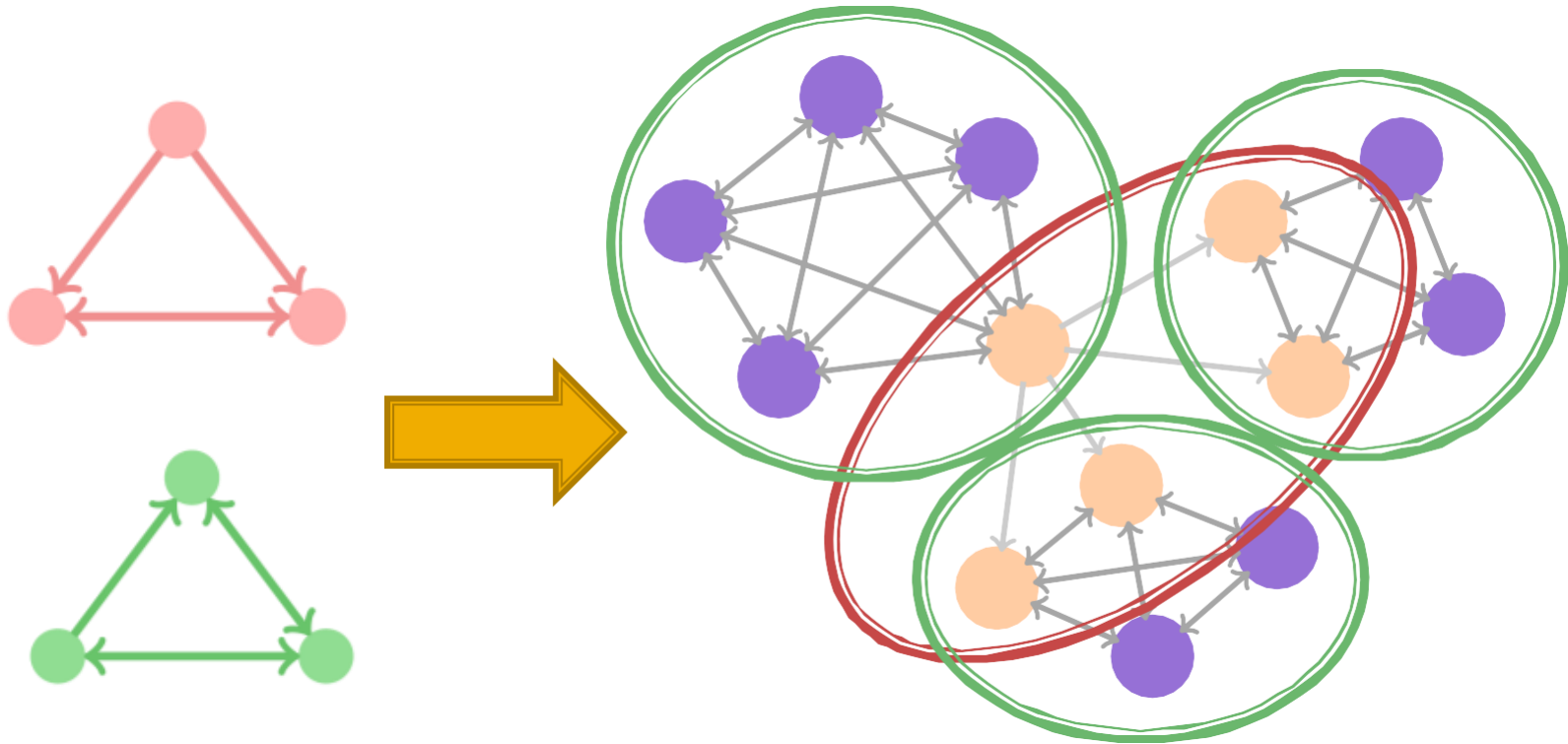
Network:



Motif:



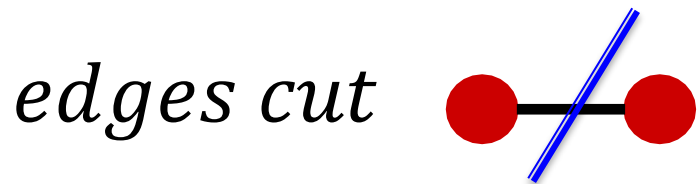
# Modules of Motifs



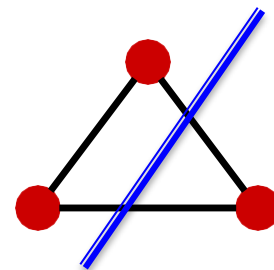
Different motifs reveal different  
modular structures!

# Defining Motif Conductance

Generalize Cut and Volume to motifs:



*motifs cut*



$vol(S) = \#(\text{edge end-points in } S)$



$vol_M(S) = \#(\text{motif end-points in } S)$

$$\phi(S) = \frac{\#(\text{edges cut})}{vol(S)}$$

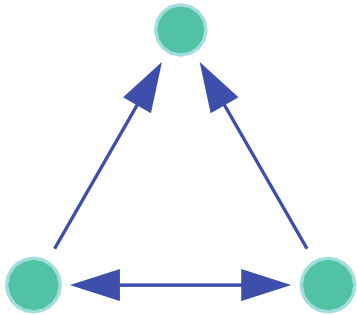


$$\phi(S) = \frac{\#(\text{motifs cut})}{vol_M(S)}$$

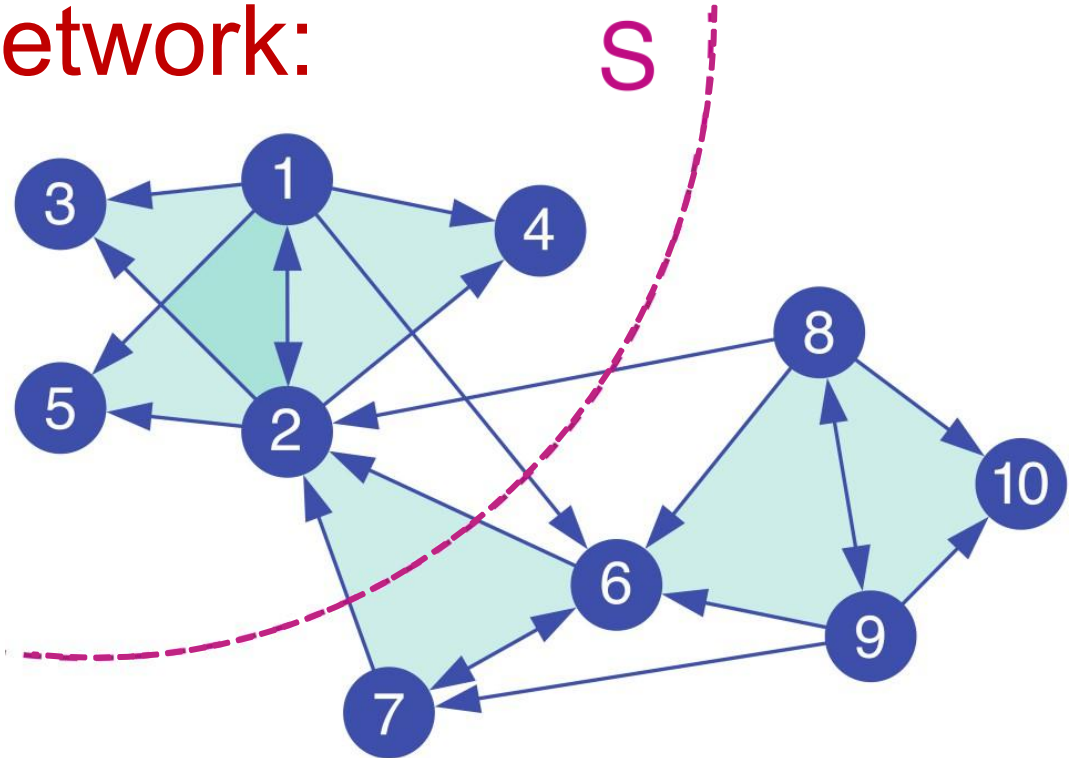
[Benson, Gleich, Leskovec, Science, 2016]

# Motif Conductance: Example

Motif:



Network:



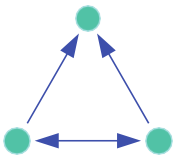
$$M(S) = \frac{\text{motifs cut}}{\text{motif volume}}$$



# Higher-order Partitioning

## How do we find clusters of motifs?

- Given a motif  $M$  and a graph  $G$
- Find a set of nodes  $S$  that minimizes motif conductance

$$M(S) = \frac{\text{motifs cut}}{\text{motif volume}}$$


**Bad news:** Finding set  $S$  with the minimal motif conductance is NP-hard!

# Motif Spectral Clustering

## **Solution:** Motif Spectral Clustering

- Input: Graph  $G$  and motif  $M$
- Using  $G$  form a new weighted graph  $W^{(M)}$
- Apply spectral clustering on  $W^{(M)}$
- Output the clusters

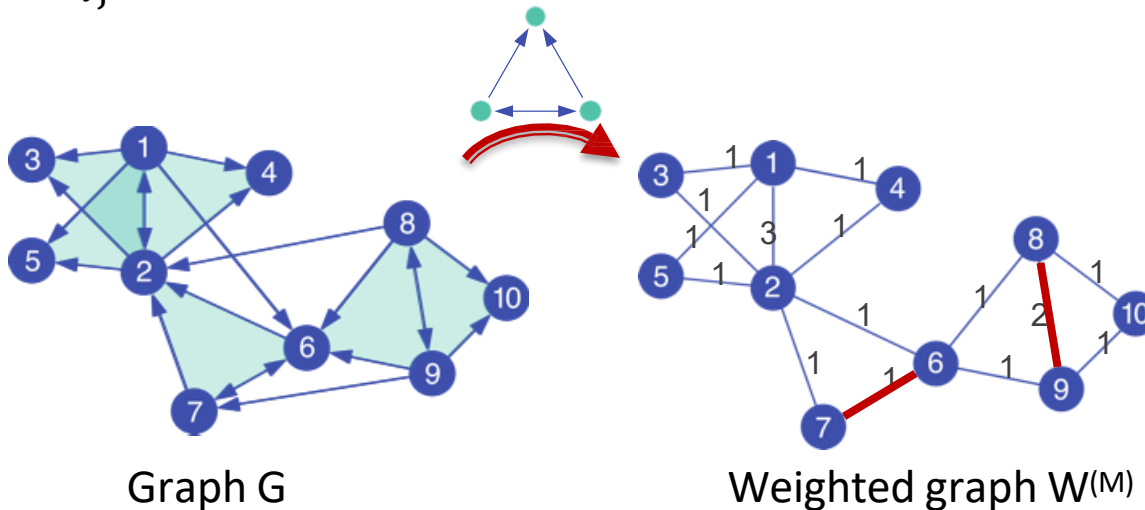
**Theorem:** Resulting clusters will obtain near optimal motif conductance

# Optimizing Motif Conductance

- **Three steps:**

- **1) Pre-processing**

- $W_{ij}^{(M)} = \# \text{ times edge } (i, j) \text{ participates in the motif } M$



See lecture 5 on motifs and the ESU algorithm for enumerating them

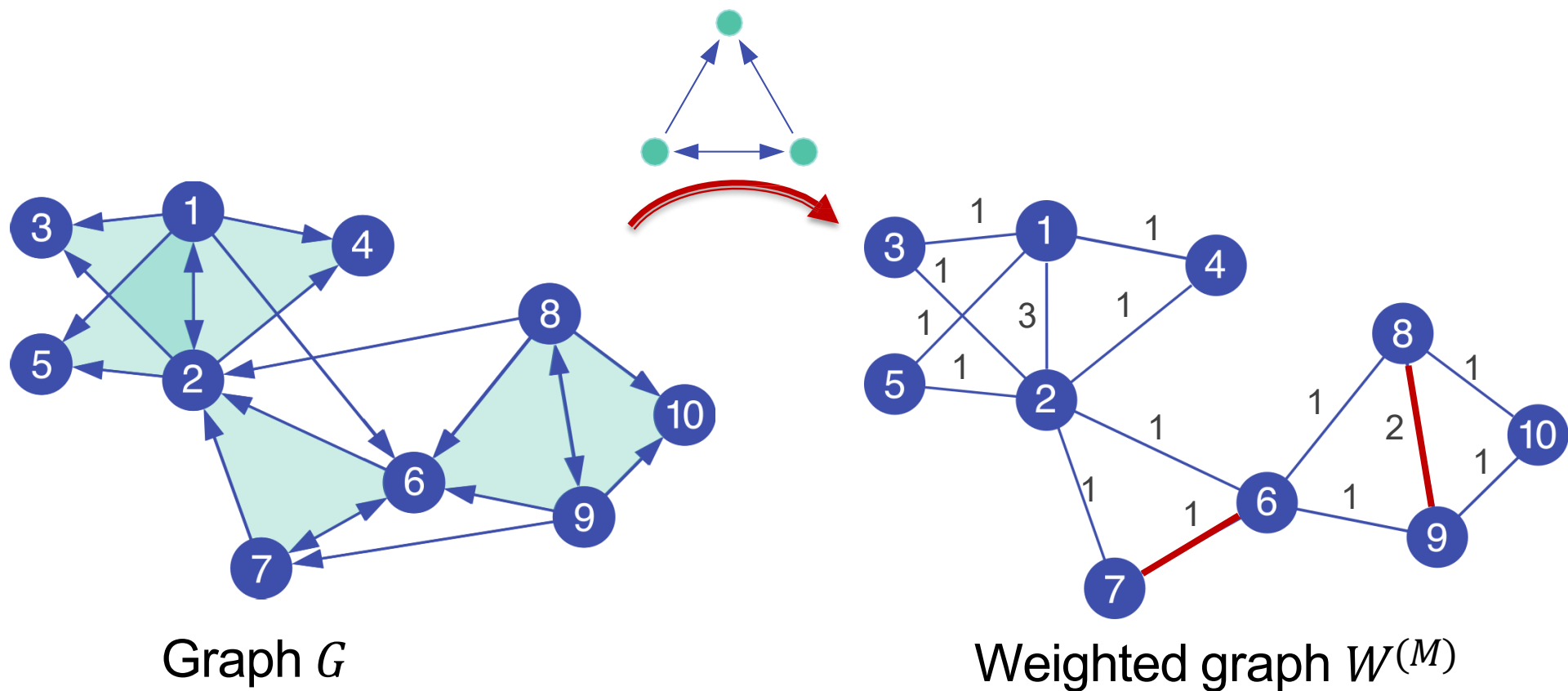
- **2) Decomposition**

- Use standard spectral clustering (but on  $W^{(M)}$ )

- **3) Grouping**

- Same as standard spectral clustering

# Motif Spectral Clustering

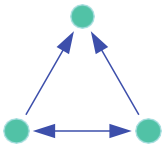


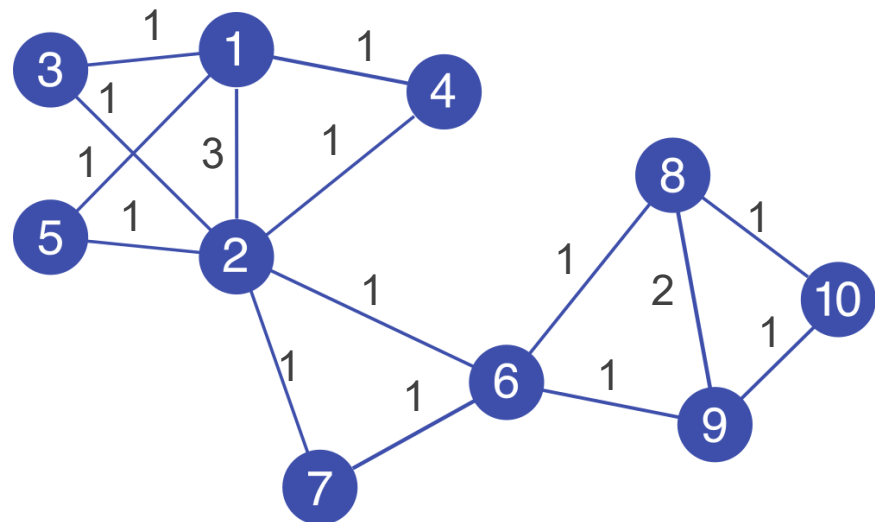
$$W_{ij}^{(M)} = \# \text{ of times edge } (i,j) \text{ participates in motif } M$$

# Motif Spectral Clustering

## Insight:

Spectral clustering on weighted graph  $W^{(M)}$  finds clusters of low motif conductance:

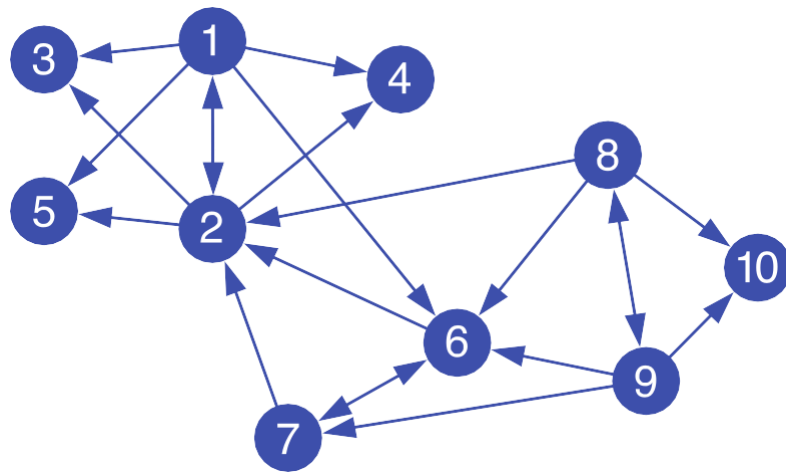
$$M(S) = \frac{\text{motifs cut}}{\text{motif volume}}$$




Weighted graph  $W^{(M)}$

$W_{ij}^{(M)} = \# \text{ of times edge } (i,j) \text{ participates in motif } M$

# Step 1: Create W



Nodes

|    | 1        | 2        | 3 | 4 | 5 | 6 | 7 | 8        | 9        | 10 |
|----|----------|----------|---|---|---|---|---|----------|----------|----|
| 1  | 0        | <b>3</b> | 1 | 1 | 1 | 0 | 0 | 0        | 0        | 0  |
| 2  | <b>3</b> | 0        | 1 | 1 | 1 | 1 | 1 | 0        | 0        | 0  |
| 3  | 1        | 1        | 0 | 0 | 0 | 0 | 0 | 0        | 0        | 0  |
| 4  | 1        | 1        | 0 | 0 | 0 | 0 | 0 | 0        | 0        | 0  |
| 5  | 1        | 1        | 0 | 0 | 0 | 0 | 0 | 0        | 0        | 0  |
| 6  | 0        | 1        | 0 | 0 | 0 | 0 | 1 | 1        | 1        | 0  |
| 7  | 0        | 1        | 0 | 0 | 0 | 1 | 0 | 0        | 0        | 0  |
| 8  | 0        | 0        | 0 | 0 | 0 | 1 | 0 | 0        | <b>2</b> | 1  |
| 9  | 0        | 0        | 0 | 0 | 0 | 1 | 0 | <b>2</b> | 0        | 1  |
| 10 | 0        | 0        | 0 | 0 | 0 | 0 | 0 | 1        | 1        | 0  |

**Step 1:** Given motif M. Form weighted graph  $W^{(M)}$

# Step 2: Apply Spectral Clust to $W^{(M)}$

**Step 2:** Apply spectral clustering:  
Compute Fiedler vector  $x$   
associated with  $\lambda_2$  of the Laplacian of  $L^{(M)}$

$$\text{Set: } L^{(M)} = D^{(M)} - W^{(M)}$$

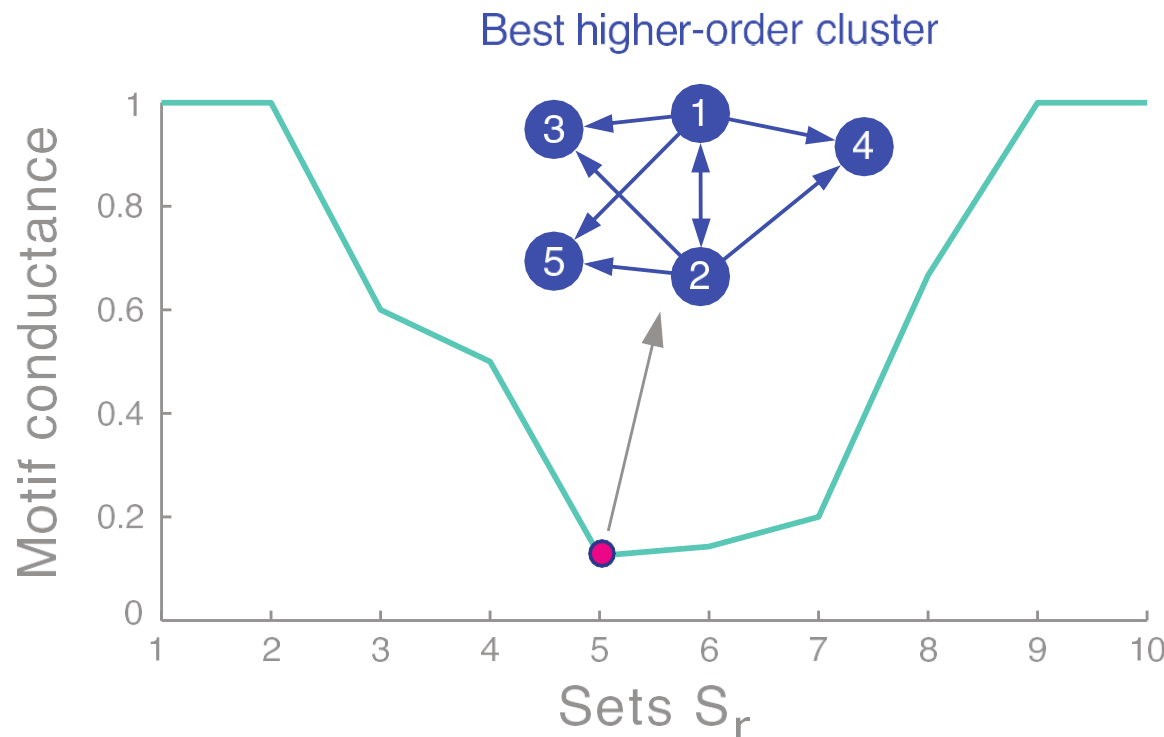
$$\text{Compute: } L^{(M)}x = \lambda_2 x$$

Use  $x$  to identify communities

Degree matrix

$$D_{ii}^{(M)} = \sum_j W_{ij}^{(M)}$$

# Step 3: Grouping (Sweep Procedure)



**Step 3:** Sort nodes by their values in  $x$ :  $x_1, x_2, \dots, x_n$

Let  $S_r = \{x_1, \dots, x_r\}$  and compute the motif conductance of each  $S_r$



# Motif Cheeger Inequality

**Theorem:** The algorithm finds a set of nodes  $S$  for which

$$M(S) \leq 4 \frac{q}{M^*}$$

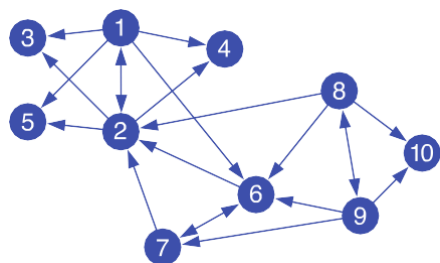
$M(S)$ ... motif conductance of  $S$  found by our algorithm

$M^*$  ... motif conductance of optimal set  $S^*$

**In other words:** Clusters  $S$  found by the method are provably **near optimal**

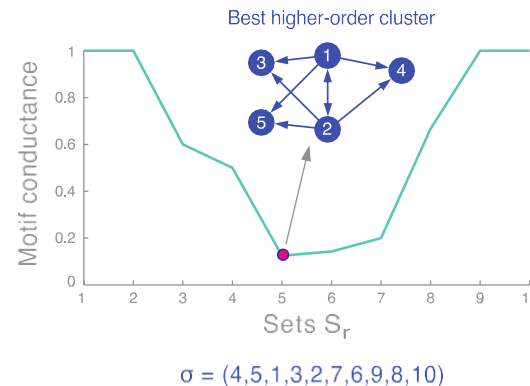
# Summary

- Generalization of community detection to higher-order structures
- Motif-conductance objective admits a motif Cheeger inequality
- Simple, fast, and scalable:



Nodes

|    | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|----|---|---|---|---|---|---|---|---|---|----|
| 1  | 0 | 3 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0  |
| 2  | 3 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0  |
| 3  | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  |
| 4  | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  |
| 5  | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  |
| 6  | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 0  |
| 7  | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0  |
| 8  | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 2 | 1  |
| 9  | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 2 | 0 | 1  |
| 10 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0  |



# Two Examples

## 1) We don't know a motif of interest

- Food webs and new applications

## 2) We know the motif of interest

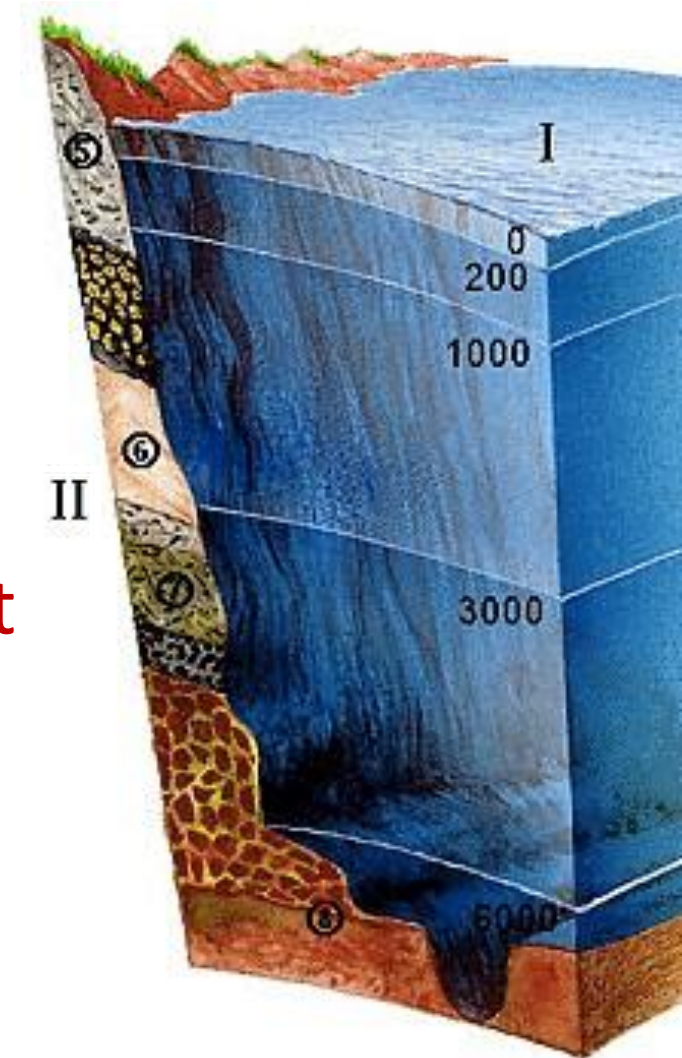
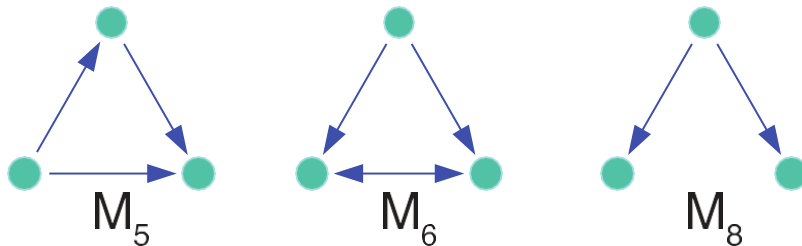
- Regulatory transcription networks, connectome, social networks

# Application 1: Food webs

## Florida Bay food web:

- **Nodes:** species in the ecosystem
- **Edges:** carbon exchange (who eats whom)

Different motifs capture different energy flow patterns:



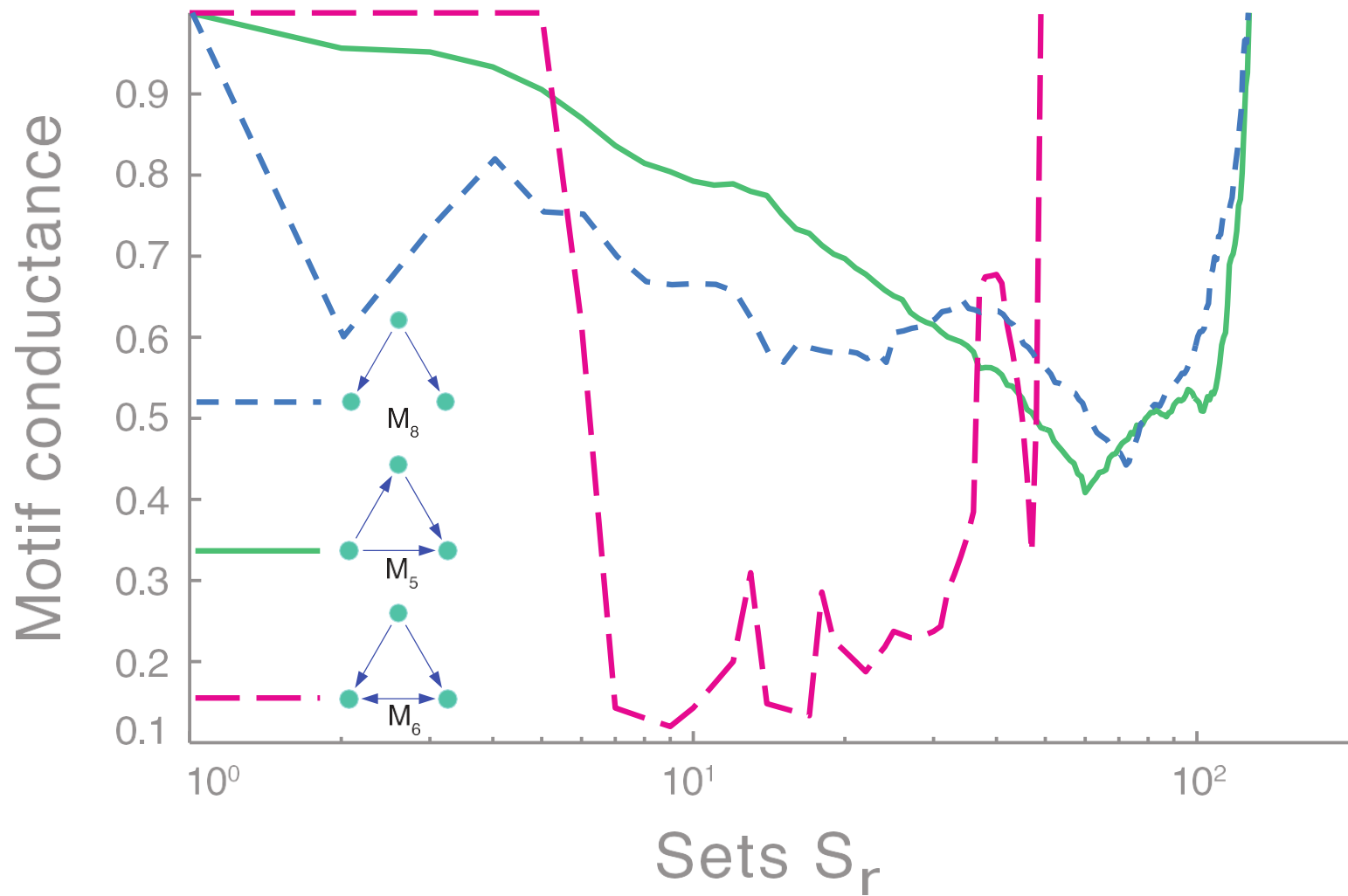
# Florida Bay Food Web

## Which motif organizes the food web?

Approach:

- Run motif spectral clustering separately for each of the 13 motifs
- Examine the **Sweep profile (next slide)** to see which motif gives best clusters

# Florida Bay Food Web

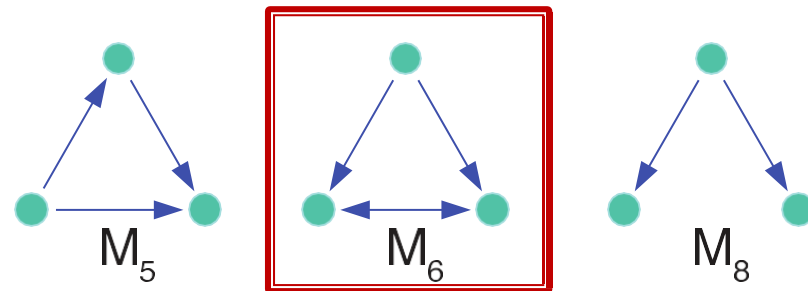
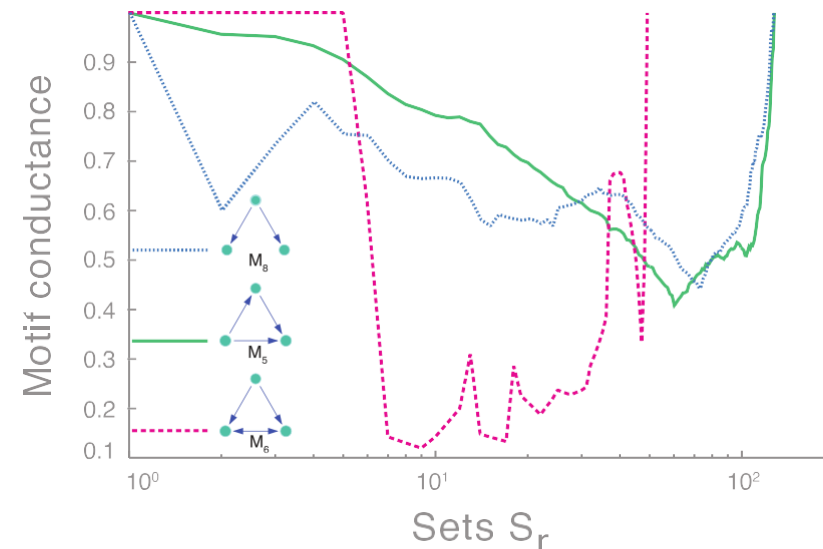


# Food Web: Observations

Observation:

Network organizes  
based on motif  $M_6$  (but  
not  $M_5$  or  $M_8$ )

- There exist good cuts for  $M_6$  but not for  $M_5$  or  $M_8$



# Food Web: Clusters

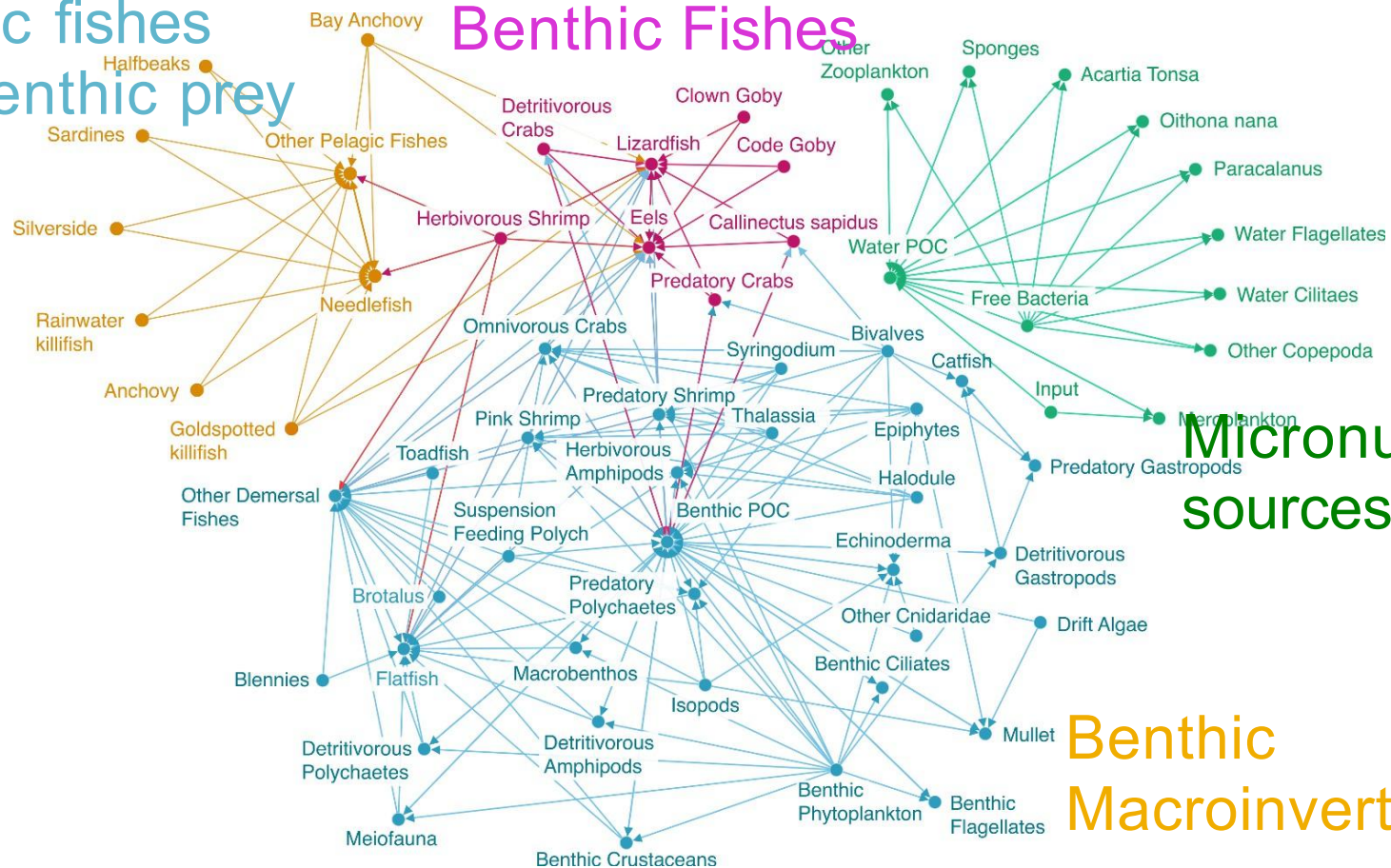
Pelagic fishes  
and benthic prey

Benthic Fishes

Micronutrient  
sources

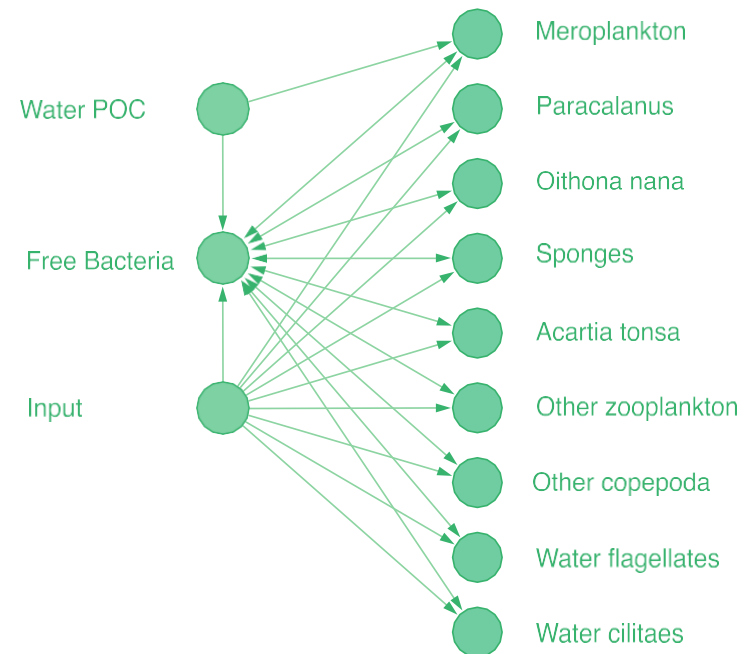
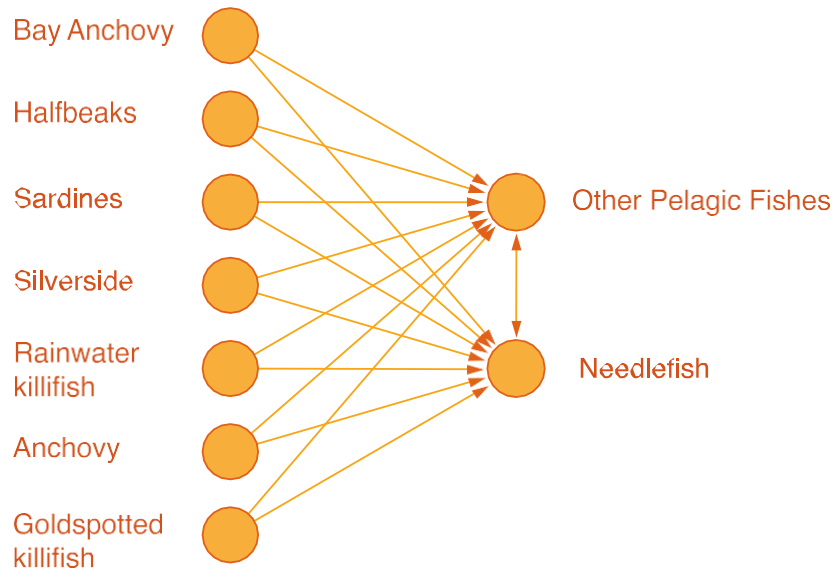
Benthic  
Macroinvertebrates

$M_6$  reveals known aquatic layers with  
higher accuracy (84% vs. 65%)



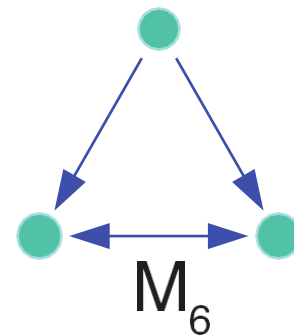


# Structure of Aquatic Layers



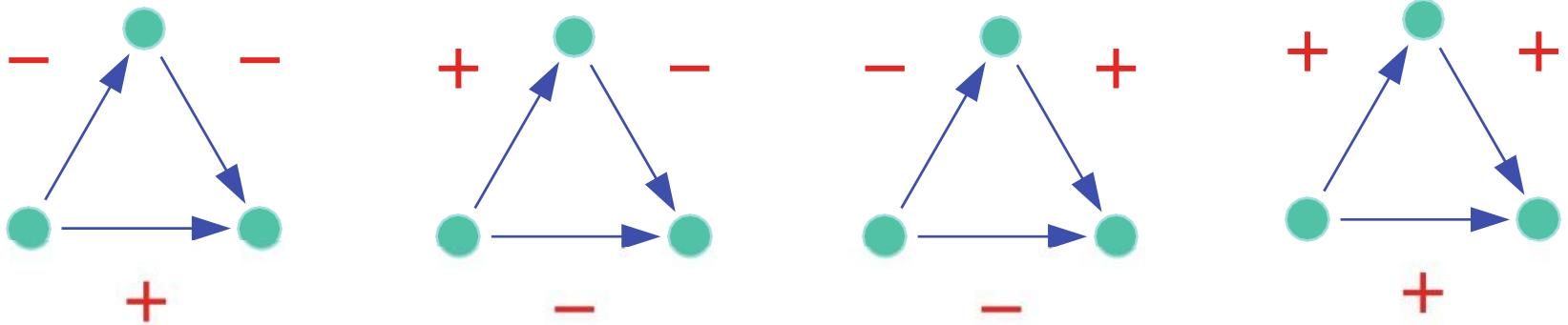
Aquatic layers organize based on  $M_6$

- Many instances of  $M_6$  inside
- Few instances of  $M_6$  cross



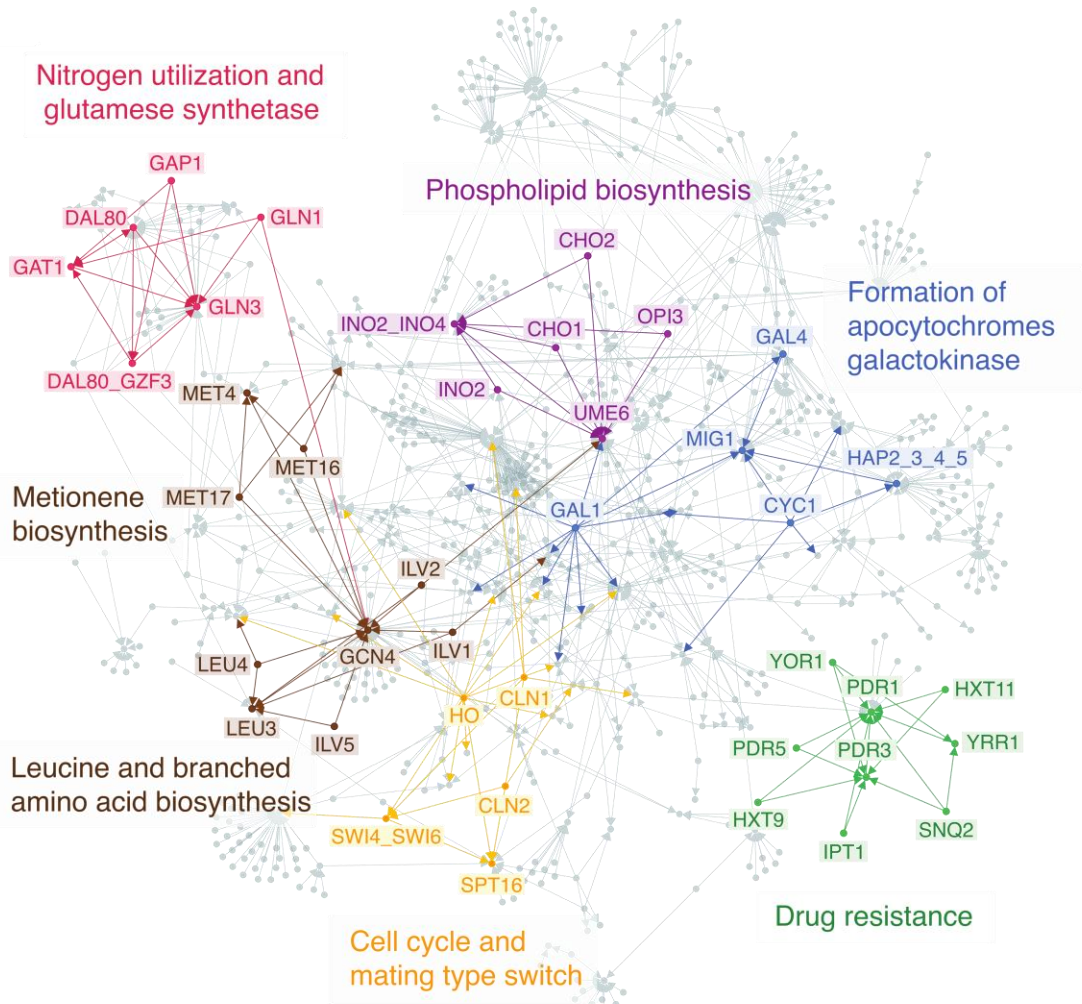
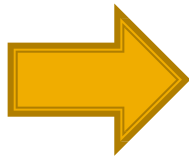
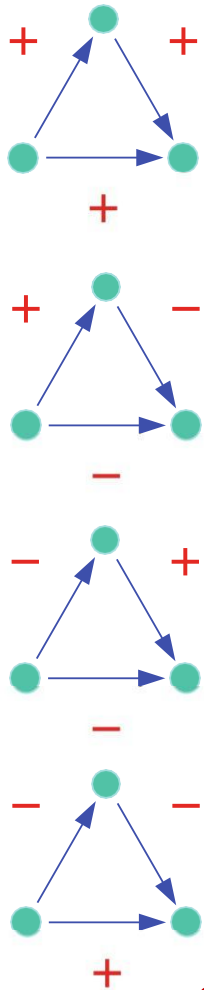
## (2) Gene Regulatory Networks

- Nodes are groups of genes in mRNA
- Edges are directed transcriptional regulation relationships



- The “feedforward loop” motif represents biological function [Alon ‘07]

# Yeast Regulatory Network

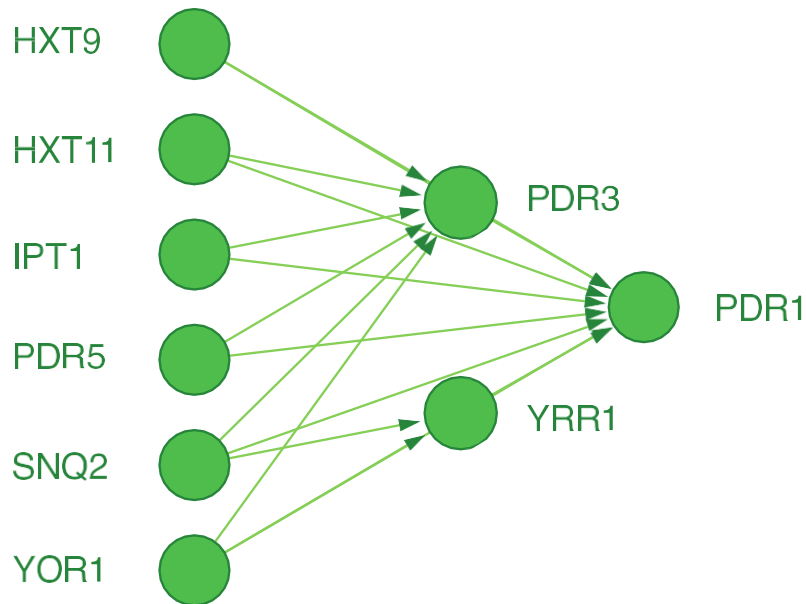


**97% detection accuracy (vs. 68-82%)**

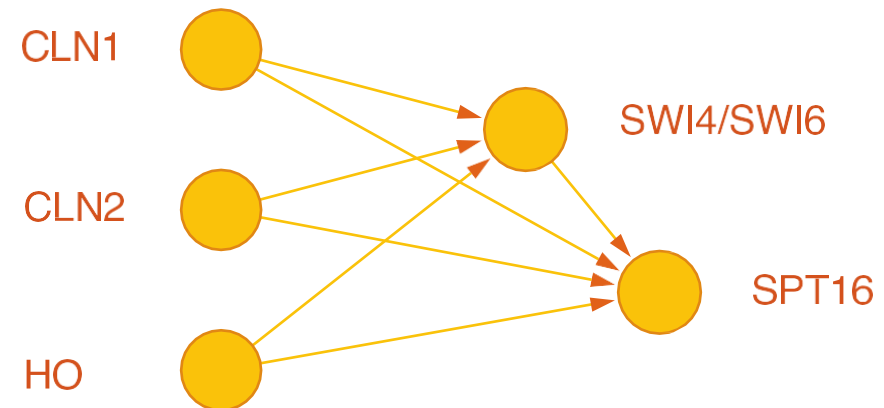
# Structure of Modules

## ■ Feed forward loops:

Drug resistance



Cell cycle and mating type switch



# Many other partitioning methods

## ■ METIS:

- Heuristic but works really well in practice
- <http://glaros.dtc.umn.edu/gkhome/views/metis>

## ■ Graclus:

- Based on kernel k-means
- <http://www.cs.utexas.edu/users/dml/Software/gracclus.html>

## ■ Louvain:

- Based on Modularity optimization
- <http://perso.uclouvain.be/vincent.blondel/research/louvain.html>

## ■ Clique percorlation method:

- For finding overlapping clusters
- <http://angel.elte.hu/cfinder/>